

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS FLORIANÓPOLIS
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Lucas Bueno Cesario

POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS

Florianópolis
2026

Lucas Bueno Cesario

POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS

Trabalho de Conclusão de Curso submetido ao Curso de Graduação em Ciência da Computação do Centro Tecnológico da Universidade Federal de Santa Catarina para obtenção do título de Bacharel em Ciência da Computação.
Orientador: Prof. Ronaldo dos Santos Mello, Dr.

Florianópolis
2026

Ficha catalográfica para trabalhos acadêmicos.

Cesario, Lucas Bueno
Polyglot Import CSV : Uma solução para importação de documentos CSV para múltiplos bancos de dados / Lucas Bueno Cesario ; orientador, Ronaldo dos Santos Mello, 2026.
49 p.

Trabalho de Conclusão de Curso (graduação) - Universidade Federal de Santa Catarina, Centro Tecnológico, Graduação em Ciências da Computação, Florianópolis, 2026.

Inclui referências.

1. Ciências da Computação. 2. Persistência poliglota. 3. Importação de dados CSV. 4. NoSQL. 5. Software Livre. I. Mello, Ronaldo dos Santos. II. Universidade Federal de Santa Catarina. Graduação em Ciências da Computação. III. Título.

Lucas Bueno Cesario

POLYGLOT IMPORT CSV: UMA SOLUÇÃO PARA IMPORTAÇÃO DE DOCUMENTOS CSV PARA MÚLTIPLOS BANCOS DE DADOS

Este Trabalho de Conclusão de Curso foi julgado adequado para obtenção do Título de "Bacharel em Ciência da Computação" e aprovado em sua forma final pelo Curso de Graduação em Ciências da Computação.

Florianópolis, _____ de _____ de 2026.

_____

Profa. Lúcia Helena Martins Pacheco, Dra.
Coordenadora do Curso

Banca Examinadora:

_____

Prof. Ronaldo dos Santos Mello, Dr.
Orientador
Universidade Federal de Santa Catarina

_____

Profa. Carina Friedrich Dorneles, Dra.
Avaliadora
Universidade Federal de Santa Catarina

_____

Prof. Jônata Tyska Carvalho, Dr.
Avaliador
Universidade Federal de Santa Catarina

RESUMO

A persistência poliglota é essencial na arquitetura contemporânea de bancos de dados, integrando diversas tecnologias de armazenamento em uma mesma aplicação. A ferramenta *Polyglot Import CSV* destaca-se como solução para otimizar a importação de dados CSV em diversos bancos de dados simultaneamente, superando desafios da persistência poliglota. Optou-se pelo formato CSV por ser amplamente utilizado na importação de dados pelos diversos SGBDs e por ser um formato simples e familiar tanto a humanos quanto a sistemas. Por meio de configurações declarativas em arquivos JSON, a ferramenta oferece suporte ao principal representante de código aberto dos bancos de dados relacionais — o PostgreSQL — e aos principais representantes dos bancos de dados NoSQL: Redis (chave-valor), MongoDB (documento), Apache Cassandra (colunar) e Neo4j (grafo). A importação admite dois modos de entrada — um arquivo CSV por entidade ou um único CSV combinado — e realiza a conversão nativa aos tipos de cada SGBD, com observabilidade e métricas por fase de execução. São oferecidas duas estratégias de desempenho: a escrita em lote otimizada e um modelo de execução em fluxo, de memória limitada, capaz de importar arquivos maiores que a memória disponível. Uma avaliação empírica com dados reais quantificou esses ganhos: a escrita otimizada acelerou a importação em cerca de nove vezes e o modo de fluxo reduziu o pico de memória em mais de doze vezes no maior volume medido, mantendo-o constante em relação ao tamanho do arquivo. Não foram identificadas ferramentas equivalentes que realizem, a partir de arquivos CSV e de regras declarativas, a importação simultânea para múltiplos paradigmas de dados, o que evidencia a originalidade da proposta. A publicação do código aberto da ferramenta impulsiona avanços colaborativos na dinâmica arquitetura moderna de bancos de dados.

Palavras-chave: Persistência poliglota. Ferramenta *Polyglot Import CSV*. Importação de dados CSV. Avaliação de desempenho. JSON. SQL. NoSQL. Software Livre.

ABSTRACT

Polyglot persistence is essential in contemporary database architecture, integrating various storage technologies within the same application. The Polyglot Import CSV tool stands out as a solution for optimizing CSV data import into multiple databases simultaneously, overcoming polyglot persistence challenges. The CSV format was chosen because it is widely used for data import by various DBMSs and because it is a simple and familiar format for both humans and systems. Through declarative configurations in JSON files, the tool supports the leading open-source representative of relational databases — PostgreSQL — and the main representatives of NoSQL databases: Redis (key-value), MongoDB (document), Apache Cassandra (columnar), and Neo4j (graph). The import supports two input modes — one CSV file per entity or a single combined CSV — and performs native type casting to each DBMS, with observability and per-phase execution metrics. Two performance strategies are offered: optimized batch writing and a streaming execution model, with bounded memory, able to import files larger than available memory. An empirical evaluation with real data quantified these gains: optimized writing sped up the import by roughly nine times, and the streaming mode reduced peak memory by more than twelve times at the largest measured volume, keeping it constant with respect to file size. No equivalent tools were identified that perform, from CSV files and declarative rules, the simultaneous import to multiple data paradigms, which highlights the originality of the proposal. The open-source publication of the tool drives collaborative advances in the dynamic modern database architecture.

Keywords*: Polyglot persistence. Polyglot Import CSV tool. CSV data import. Performance evaluation. JSON. SQL. NoSQL. Free Software.*

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce. 17
Figura 2 – Diferentes tipos de modelos de dados NoSQL. 18
Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis). 19
Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra). 20
Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB). 21
Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j). . 22
Figura 7 – Visão geral da ferramenta: entradas, processamento e saídas. . . . 30
Figura 8 – Estrutura do JSON Schema de importação (`import_config`). 34
Figura 9 – Estrutura do JSON Schema de conexão (`sgbd_config`). 34
Figura 10 – Fluxo de carregamento e validação das entradas da ferramenta. . . 44
Figura 11 – Algoritmo de execução da importação PolyglotImportCSV. 45
Figura 12 – Tempo total de importação por estratégia de escrita ($N = 10\,000$, materialização; eixo vertical em escala logarítmica). Campanha sem instrumentação de memória. 61
Figura 13 – *Throughput* de escrita por SGBD ($N = 100\,000$, multifonte, materialização, estratégia otimizada). Campanha sem instrumentação de memória. 63
Figura 14 – Pico de memória por modelo de execução em função do volume (multifonte, escala logarítmica): a materialização cresce com o arquivo; o fluxo achata. 64
Figura 15 – Tempo total por modelo de execução em função do volume (multifonte): o sobrecusto do fluxo encolhe de $+61\%$ para $+10\%$ à medida que a escrita passa a dominar. 65

LISTA DE TABELAS

Tabela 1 – Campos de mapeamento de coluna (`columnSpec`). 36
Tabela 2 – Correspondência entre tipo inferido e tipo de destino na geração de DDL. 37
Tabela 3 – Linhas da fonte `stock` (`ecommerce_stock.csv`, colunas de identificação resumidas). 48
Tabela 4 – Relação `inventory` materializada no PostgreSQL a partir da fonte `stock`. 49
Tabela 5 – Partição do usuário `user1` na tabela `user_activity_log` (Cassandra). 50
Tabela 6 – Materialização de uma linha da fonte `stock` nos diferentes SGBDs. . 52
Tabela 7 – Contagens reportadas pelo modo `--dry-run` no cenário e-commerce. 53
Tabela 8 – Imagens Docker dos SGBDs utilizadas na avaliação. 60
Tabela 9 – Tempo total e das fases dominantes por estratégia de escrita, em segundos ($N = 10\,000$, materialização). Campanha sem instrumentação de memória. 61
Tabela 10 – Tempo por fase de uma importação otimizada ($N = 100\,000$, multifonte, materialização), em segundos. Campanha sem instrumentação de memória. 62
Tabela 11 – Escrita por SGBD em uma importação otimizada ($N = 100\,000$, multifonte, materialização). Campanha sem instrumentação de memória. 62
Tabela 12 – Multifonte *versus* combinado ($N = 100\,000$, estratégia otimizada). Tempos e picos de memória provêm de campanhas distintas, conforme indicado. 63
Tabela 13 – Pico de memória por modelo de execução (N crescente, multifonte, estratégia otimizada), em MB. Campanha com instrumentação de memória. 64
Tabela 14 – Tempo total por modelo de execução (N crescente, multifonte, estratégia otimizada), em segundos. Campanha sem instrumentação de memória. 65
Tabela 15 – Pico de memória por estratégia de escrita ($N = 10\,000$, materialização), em MB. Campanha com instrumentação de memória. 66
Tabela 16 – Grupos de colunas do `ecommerce_join.csv` preenchidos por ação. . 83
Tabela 17 – Usuários do `ecommerce_join.csv` e seus dados de identificação e endereço. 84
Tabela 18 – Linhas `action == stock` do `ecommerce_join.csv`. 85
Tabela 19 – Linhas `action == purchase` do `ecommerce_join.csv`. 86
Tabela 20 – Linhas `action == select_product` do `ecommerce_join.csv`. 86

Tabela 21 – Linhas action == add_to_cart do ecommerce_join.csv. 87

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
API	*Application Programming Interface* (Interface de Programação de Aplicações)
BSON	*Binary JSON*
CI	*Continuous Integration* (Integração Contínua)
CLI	*Command Line Interface* (Interface de Linha de Comando)
CSV	*Comma-Separated Values* (Valores Separados por Vírgula)
DDL	*Data Definition Language* (Linguagem de Definição de Dados)
JSON	*JavaScript Object Notation*
NoSQL	*Not Only SQL*
SGBD	Sistema de Gerenciamento de Banco de Dados
SGPD	Sistema de Gerenciamento Poliglota de Dados
SQL	*Structured Query Language* (Linguagem de Consulta Estruturada)
TCC	Trabalho de Conclusão de Curso
UFSC	Universidade Federal de Santa Catarina
XML	*Extensible Markup Language*
YCSB	*Yahoo! Cloud Serving Benchmark*

SUMÁRIO

1 INTRODUÇÃO 13
1.1 Visão Geral 13
1.2 Objetivos 14
1.2.1 Objetivo Geral 14
1.2.2 Objetivos Específicos 14
1.3 Metodologia 14
1.4 Estrutura do Trabalho 15

2 FUNDAMENTAÇÃO TEÓRICA 16
2.1 Persistência Poliglota 16
2.2 Bancos de Dados NoSQL 18
2.2.1 Chave-valor 18
2.2.2 Colunar 19
2.2.3 Documento 20
2.2.4 Grafo 21
2.3 Bancos de Dados Multimodelo e Arquiteturas de Persistência Poliglota 22

3 TRABALHOS RELACIONADOS 24
3.1 Importação de Arquivo CSV para um Único SGBD 24
3.1.1 PostgreSQL 24
3.1.2 Redis 25
3.1.3 MongoDB 26
3.1.4 Cassandra 26
3.1.5 Neo4j 27
3.2 Importação de Arquivo CSV para SGBD Multimodelo 27
3.3 Considerações sobre os Trabalhos Relacionados 28

4 A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV 30
4.1 Visão Geral 30
4.2 Formato de Configuração (JSON Schema) 32
4.2.1 Fontes de Dados e os Dois Modos de Importação 32
4.2.2 Visão Geral da Estrutura dos Arquivos de Configuração 33
4.2.3 Raiz e Elementos Comuns 35
4.2.4 PostgreSQL 37
4.2.5 MongoDB 38

4.2.6 Apache Cassandra . . . 39
4.2.7 Redis . . . 40
4.2.8 Neo4j . . . 41
4.2.9 Validação, Consistência entre Arquivos e Fluxo de Carregamento . . . 43
4.3 Algoritmo de Execução da Importação . . . 45
4.3.1 O Cenário de Exemplo e os Arquivos de Dados . . . 47
4.3.2 A Importação em Cada SGBD . . . 48
4.3.3 Exemplo de Uso e Evidência de Execução (Cenário E-commerce) . . . 52
4.4 Observabilidade e Métricas . . . 54
4.5 Modelos de Execução e Estratégias de Escrita . . . 55
4.5.1 Estratégias de Escrita . . . 55
4.5.2 Modelos de Execução: Materialização e *Streaming* . . . 56

5 AVALIAÇÃO DE DESEMPENHO . . . 58
5.1 Metodologia . . . 58
5.1.1 Geração dos Dados . . . 59
5.1.2 Protocolo de Medição . . . 59
5.2 Configuração Experimental . . . 60
5.3 Resultados . . . 61
5.3.1 Estratégia de Escrita: Otimizada *versus* Ingênua (QP4) . . . 61
5.3.2 Decomposição por Fase e Escrita por SGBD (QP1) . . . 62
5.3.3 Modos de Importação: Multifonte *versus* Combinado (QP2) . . . 63
5.3.4 Modelos de Execução: Memória e Tempo (QP5 e QP6) . . . 64
5.3.5 Uma Observação sobre o Pico de Memória da Estratégia Otimizada 65
5.4 Análise e Discussão . . . 66
5.4.1 Ameaças à Validade . . . 67

6 ATIVIDADES FUTURAS . . . 68

7 CONSIDERAÇÕES FINAIS . . . 69

REFERÊNCIAS . . . 71

APÊNDICES 74

APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA FORMA SERIALIZADA . . . 75
A.1 Schema de Importação (`import_config.schema.json`) . . . 75
A.2 Schema de Conexão (`sgbd_config.schema.json`) . . . 79

APÊNDICE B – O ARQUIVO DE DADOS ECOMMERCE_JOIN.CSV . . . 82
B.1 Linhas com action == stock 84
B.2 Linhas com action == purchase 85
B.3 Linhas com action == select_product 86
B.4 Linhas com action == add_to_cart 87

APÊNDICE C – ARQUIVOS DE CONFIGURAÇÃO DO CENÁRIO E-COMMERCE 88
C.1 Arquivo de Importação (import_config.json) 88
C.2 Arquivo de Importação no Modo Combinado (import_config_combined.json) 91
C.3 Arquivo de Conexão (sgbd_config.json) 92

1 INTRODUÇÃO

1.1 VISÃO GERAL

Nas esferas industrial e acadêmica de bancos de dados, a concepção de que um único modelo de dados não atende a todas as necessidades tem conquistado aceitação, apontando para a perspectiva de persistência poliglota no futuro. Embora os sistemas de banco de dados relacionais devam manter sua predominância, espera-se uma convivência com diversas formas de sistemas, como os NoSQL (SADALAGE; FOWLER, 2013).

Algumas pesquisas têm sido feitas na área de persistência poliglota, como sugestões de modelos de dados unificados (CHILLóN et al., 2023), comparação de desempenho de bancos de dados multi-modelos contra abordagens de persistência poliglota (YE et al., 2023), revisões sistemáticas sobre modelagem poliglota de dados (SILVA; BORNIA; MELLO, 2024), tutoriais sobre o estado da arte e desafios abertos (KIEHN et al., 2022), e a adoção da persistência poliglota em Big Data (KHINE; WANG, 2019).

Dada essa necessidade crescente da adoção de persistência poliglota pelas aplicações, percebe-se a falta de uma ferramenta única de importação simultânea de dados para diversos SGBDs que tratem modelos de dados diferentes. Até onde se investigou (Capítulo 3), não foram identificadas ferramentas equivalentes: os importadores nativos atendem a um único SGBD por vez, e as soluções de migração ou de modelagem poliglota não realizam a materialização simultânea de um mesmo CSV em múltiplos paradigmas de dados — o que ressalta a originalidade desta proposta.

Assim sendo, este trabalho desenvolve a ferramenta *Polyglot Import CSV*, que importa dados armazenados em CSV para SGBDs relacionais e NoSQL. A configuração da ferramenta baseia-se em dois arquivos JSON: um de importação, mapeando entidades e relacionamentos em cada SGBD, considerado seu modelo de dados; outro de configuração de conexão dos SGBDs envolvidos. A escolha pelo formato JSON deve-se à sua simplicidade e ampla adoção; sua finalidade é detalhar, de forma declarativa, o mapeamento das colunas do CSV para os modelos de dados de cada SGBD, permitindo a validação prévia da importação.

CSV (*Comma-Separated Values*) é um arquivo de valores separados por vírgulas, frequentemente visualizado em Excel e outras planilhas eletrônicas. Pode haver outros tipos de valores como delimitadores, mas o mais comum é a vírgula. Muitos sistemas e processos hoje convertem seus dados para o formato CSV para saídas de

arquivo para outros sistemas, relatórios amigáveis para humanos, disponibilização de dados públicos, e outras necessidades. É um formato de arquivo padrão com o qual humanos e sistemas já estão familiarizados ao usar e manipular.

1.2 OBJETIVOS

1.2.1 Objetivo Geral

Desenvolver a ferramenta *Polyglot Import CSV* que possibilitará a importação de dados em arquivos CSV para múltiplos SGBDs: PostgreSQL, Redis, MongoDB, Cassandra ou Neo4j.

1.2.2 Objetivos Específicos

- Projetar os arquivos de configuração JSON para que eles sigam sintaxes específicas, e flexíveis para lidar com vários modelos de dados, além de adicionar opções como filtros, escolha de chave primária e escolha do nome da coluna no banco de dados;
- Não permitir configurações incorretas, ou seja, se houver erro em algum lugar dos arquivos de configuração, a importação não deve começar até que sejam corrigidos (validação prévia da configuração);
- Avaliar o desempenho da ferramenta para diferentes volumes e modelos de dados destino;
- Disponibilizar a ferramenta de forma acessível ao usuário: inicialmente por meio de uma interface de linha de comando (CLI), com possibilidade de evolução para uma interface gráfica em trabalhos futuros.

1.3 METODOLOGIA

O desenvolvimento deste trabalho segue uma metodologia de pesquisa aplicada, adotando as seguintes etapas:

1. Estudo sobre persistência poliglota e seu estado da arte;
2. Estudo sobre os paradigmas não-relacionais que serão abrangidos pela solução: chave-valor, documento, grafo e colunar;
3. Estudo de decisão de linguagem de implementação que demonstra melhor produtividade para codificar a solução;
4. Projeto dos arquivos de configuração JSON;

5. Projeto da instrução principal de importação que utiliza os arquivos de configuração;
6. Implementação da solução “Polyglot Import CSV”;
7. Testes da solução “Polyglot Import CSV” implementada;
8. Avaliação de desempenho da solução para diferentes volumes e modelos de dados destino.

Na primeira etapa, foi estudado o que é a persistência poliglota, e a sua relevância no cenário atual.

Na segunda etapa, foram estudados os diversos modelos de dados não-relacionais: como cada modelo representa suas entidades; se há relacionamentos entre entidades no modelo; se é possível a existência de entidades aninhadas e multivaloradas; as limitações de cada modelo.

Na terceira etapa, foram estudadas as formas de implementar a solução usando as linguagens Java e Python e, pela simplicidade e produtividade, a linguagem Python foi escolhida.

Na quarta e quinta etapas, foram projetados os dois arquivos de configuração JSON — um para o mapeamento da importação (entidades, relacionamentos e colunas) e outro para a conexão dos SGBDs — bem como a instrução principal de importação que os utiliza.

As etapas seguintes trataram da implementação da solução proposta, “Polyglot Import CSV”, e dos testes da mesma com diferentes volumes de dados CSV e importações simples e complexas para diferentes modelos de dados destino. A última etapa consistiu na avaliação de desempenho da ferramenta, cujos resultados — *throughput* por SGBD, ganho das estratégias de escrita otimizadas e economia de memória do modo de execução em fluxo — são apresentados no Capítulo 5.

1.4 ESTRUTURA DO TRABALHO

Este trabalho está estruturado em sete capítulos principais. O primeiro aborda os objetivos, a metodologia e o contexto que levaram à elaboração deste trabalho. O capítulo 2 revisa os fundamentos teóricos e o capítulo 3 apresenta os trabalhos relacionados. O capítulo 4 descreve a proposta da ferramenta *Polyglot Import CSV* — o formato de configuração, o algoritmo de importação, a observabilidade e os modelos de execução. O capítulo 5 apresenta a avaliação de desempenho da ferramenta, com resultados reais. O capítulo 6 discute as atividades futuras e o capítulo 7 traz as considerações finais.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 PERSISTÊNCIA POLIGLOTA

Ford (2008) introduziu o conceito de “programação poliglota” para expressar a ideia de que aplicativos devem ser desenvolvidos utilizando uma combinação de linguagens, aproveitando o fato de que diferentes linguagens são mais adequadas para lidar com diferentes problemas. Uma das consequências notáveis da programação poliglota é a transição para a “persistência poliglota”, usando a mesma analogia: pode-se dizer que múltiplas tecnologias de bancos de dados podem ser utilizadas como armazenamento persistente para diferentes tipos de casos de uso. Embora ainda exista uma quantidade considerável de dados gerenciados por sistemas relacionais, a abordagem predominante passa a ser a reflexão sobre como se deseja manipular os dados antes de determinar qual tecnologia é a mais adequada para essa manipulação (FOWLER, 2011).

A persistência poliglota abrange uma variedade de formatos de dados, incluindo estruturados, semi-estruturados e não estruturados. Dados estruturados geralmente englobam formatos como relacional, objeto-relacional, e grafos de propriedades. Dados semi-estruturados incluem principalmente documentos JSON e XML. Por fim, dados não estruturados são representados por arquivos de texto, imagens e áudios, dentre outros (YE et al., 2023).

Um domínio em que a persistência poliglota pode ser aplicada é o de um e-commerce. Nesse cenário, diferentes categorias de dados possuem características e padrões de acesso distintos — volume de escrita, frequência de leitura, necessidade de consistência, estrutura dos registros — o que torna mais adequado o uso de um SGBD específico para cada uma delas. A Figura 1 ilustra esse cenário, mapeando cada categoria de dado do e-commerce ao SGBD mais adequado, conforme sugerido a seguir:

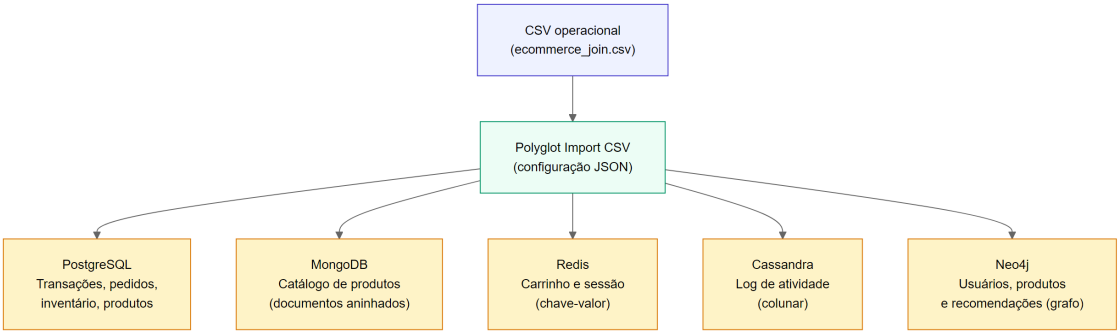


Figura 1 – Exemplo de aplicação da Persistência Poliglota em um e-commerce.
Fonte: Elaborado pelo autor (2026).

- **Dados Financeiros/Transacionais de Pagamento e Inventário de Produtos:** Pedidos, pagamentos e estoques exigem garantias transacionais fortes (ACID) para evitar inconsistências — por exemplo, debitar um pagamento sem registrar o pedido correspondente. Além disso, consultas analíticas sobre esses dados frequentemente combinam várias entidades por meio de junções. Bancos de dados relacionais como o PostgreSQL são a escolha natural para esse perfil, pois oferecem suporte nativo a transações com consistência forte, integridade referencial e uma linguagem de consulta expressiva (SQL).

- **Dados do Catálogo de Produtos:** O catálogo de um e-commerce tende a ter esquema variável: diferentes produtos possuem atributos distintos (um livro físico com ISBN e editora; um eletrodoméstico com voltagem e garantia). Além disso, o catálogo é lido com muito maior frequência do que escrito, e cada consulta normalmente recupera o produto completo de uma vez. Bancos de dados orientados a documentos como o MongoDB são otimizados exatamente para esse perfil: armazenam cada produto como um documento autocontido (favorecendo leituras de agregados complexos), suportam esquemas flexíveis e oferecem índices secundários e *pipelines* de agregação para fins de consulta.

- **Dados da Sessão e do Carrinho de Compras do Usuário:** Dados de sessão e carrinho são acessados a cada interação do usuário durante a navegação, exigindo latência mínima, e são também inerentemente temporários — um carrinho abandonado não precisa persistir indefinidamente. Bancos de dados chave-valor em memória como o Redis oferecem latência de microssegundos, expiração automática de chaves e são horizontalmente escaláveis para absorver picos de tráfego, tornando-se a escolha ideal para dados de natureza efêmera e de acesso intenso.

- **Dados de Atividade do Usuário:** Eventos de navegação (*clickstream*), visualizações de produto e histórico de ações são gerados em alto volume e alta

velocidade. Consultas típicas percorrem o histórico de um usuário em um intervalo de tempo, ou agregam eventos por tipo de ação. Bancos de dados colunares como o Apache Cassandra são projetados para ingestão massiva com particionamento por chave primária composta (por exemplo, `(user_id, timestamp)`), garantindo escritas rápidas e leituras eficientes por usuário ou período — padrão ideal para séries temporais de atividade.

- **Dados de Recomendação:** Sistemas de recomendação dependem de relacionamentos entre entidades: usuários que compraram os mesmos produtos, produtos frequentemente adquiridos em conjunto ou usuários com perfis semelhantes. Esses padrões são naturalmente modelados como grafos, onde nós representam usuários e produtos, e arestas representam interações (comprou, avaliou, visualizou, etc.). Bancos de dados orientados a grafos como o Neo4j permitem percorrer esses relacionamentos com eficiência por meio de consultas Cypher, tarefa que em um banco de dados relacional exigiria múltiplas junções e difícil otimização.

2.2 BANCOS DE DADOS NOSQL

Os bancos de dados NoSQL podem ser divididos em subcategorias com base em seus modelos de dados. Este trabalho utiliza a classificação de Hecht e Jablonski (2011), que categoriza os bancos de dados NoSQL em quatro tipos: chave-valor, colunares, documentos e grafos.

A Figura 2 ilustra esses modelos de dados, que são detalhados a seguir.



Figura 2 – Diferentes tipos de modelos de dados NoSQL.
Fonte: Elaborado pelo autor (2026).

2.2.1 Chave-valor

Os bancos de dados de chave-valor têm um modelo de dados simples baseado justamente em pares chave-valor, semelhante a um mapa associativo ou dicionário. A chave identifica exclusivamente o valor e é usada para armazenar e recuperar o valor no banco de dados, funcionando como uma chave primária. O valor pode ser usado para armazenar qualquer dado arbitrário, incluindo um número inteiro, uma *string*, um *array* ou um objeto, proporcionando uma representação de dados sem esquema conhecido.

Além disso, os bancos de dados de chave-valor são muito eficientes no armazenamento de dados distribuídos, mas não são adequados para cenários que requerem a definição de relacionamentos entre dados ou esquemas fortemente estruturados.

Qualquer funcionalidade que necessite desses requisitos deve ser implementada na aplicação cliente que interage com o banco de dados chave-valor.

Como os valores são opacos para os bancos de dados, esses não podem lidar com consultas e indexações a nível de dados, podendo realizar consultas apenas por meio de chaves. Um exemplo de SGBD chave-valor é o Redis, que mantém os dados tanto na memória como no disco. A Figura 3 ilustra o modelo com um exemplo de carrinho de compras e sessão de usuário, em que cada chave aponta para um valor opaco ao SGBD.

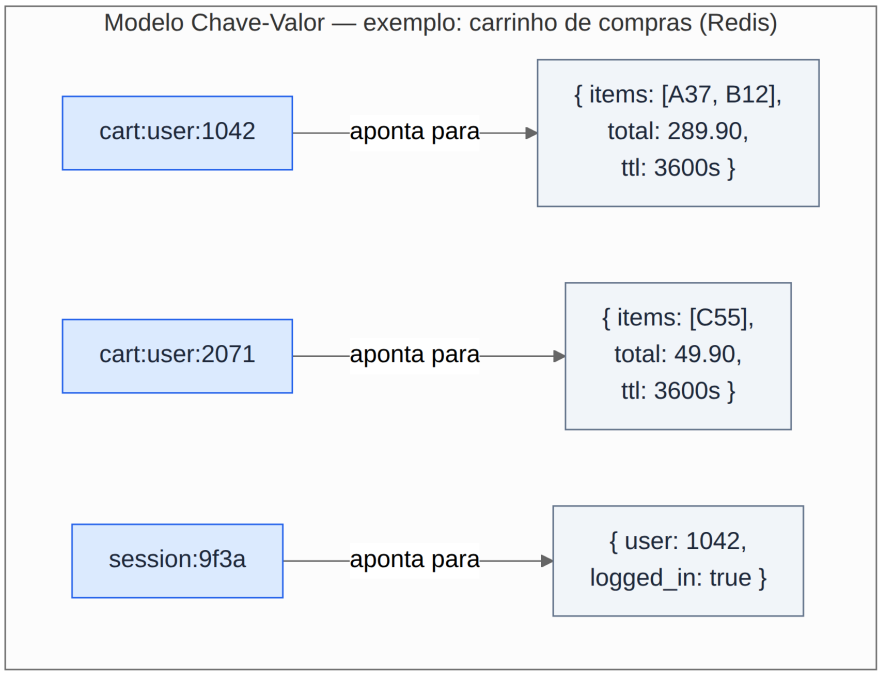


Figura 3 – Modelo chave-valor: chaves apontando para valores arbitrários (Redis).
Fonte: Elaborado pelo autor (2026).

2.2.2 Colunar

Em bancos de dados colunares, o conjunto de dados consiste em várias linhas, cada uma identificada por uma chave de linha única, semelhante a uma chave primária. Cada linha é composta por uma família (ou conjunto) de colunas, e diferentes linhas podem ter diferentes famílias de colunas. Semelhante aos bancos de dados de chave-valor, a chave de linha funciona como a chave, e o conjunto de famílias de colunas atua como o valor representado pela chave de linha. No entanto, colunas em uma família de colunas consistem em pares atributo-valor que podem ser indexadas. O SGBD Cassandra oferece a funcionalidade adicional de supercolunas, que são criadas agrupando várias colunas.

Normalmente, os dados pertencentes a uma linha são armazenados juntos no mesmo nó do servidor. No entanto, o Cassandra pode distribuir uma única linha

por vários nós de servidor usando chaves de partição compostas. Nos bancos de dados colunares, a configuração das famílias de colunas é tipicamente feita durante a inicialização. No entanto, a definição prévia de colunas não é necessária, oferecendo grande flexibilidade no armazenamento de qualquer tipo de dado.

De forma semelhante aos bancos de dados de chave-valor, qualquer lógica que exija relações deve ser implementada na aplicação cliente, ou seja, essa categoria de banco de dados também não têm ciência de relacionamentos entre dados e controle de integridade referencial. A Figura 4 apresenta um exemplo de *log* de atividade, em que cada chave de linha (`user_id`) agrupa uma família de colunas com eventos indexados por *timestamp*.

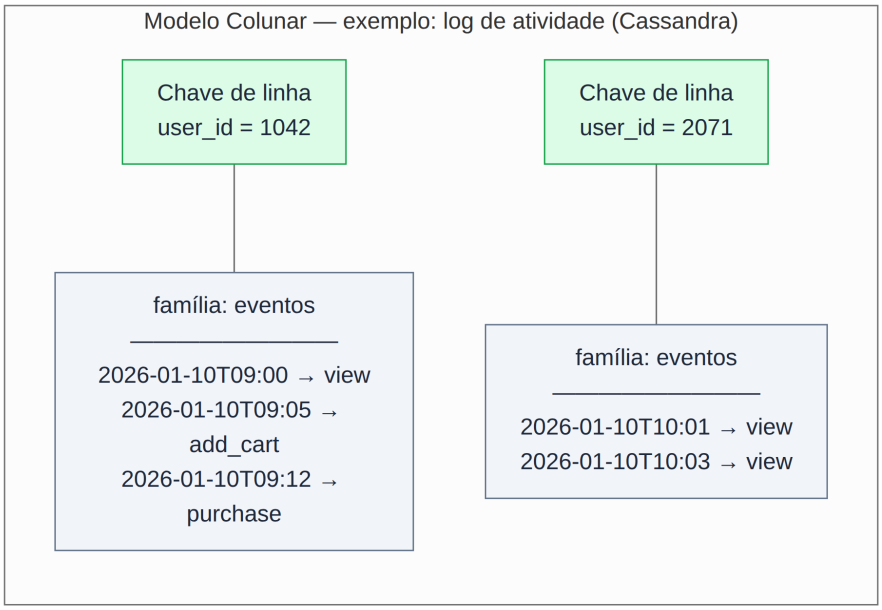


Figura 4 – Modelo colunar: chaves de linha e famílias de colunas (Cassandra).
Fonte: Elaborado pelo autor (2026).

2.2.3 Documento

Bancos de dados orientados a documento armazenam dados como documentos autocontidos, em geral em formato JSON ou BSON. Cada documento possui um identificador e um conjunto de campos que podem variar entre documentos de uma mesma coleção, favorecendo esquemas flexíveis e agregados de leitura (*read-optimized*). O SGBD MongoDB é um exemplo popular nesta categoria. Ele define coleções de documentos, índices secundários e *pipelines* de agregação, permitindo consultas ricas sem exigir junções (*joins*) no servidor. A Figura 5 ilustra uma coleção de catálogo de produtos, em que cada documento é autocontido e pode conter subdocumentos aninhados (como `category` e `stock`) e campos distintos entre si.

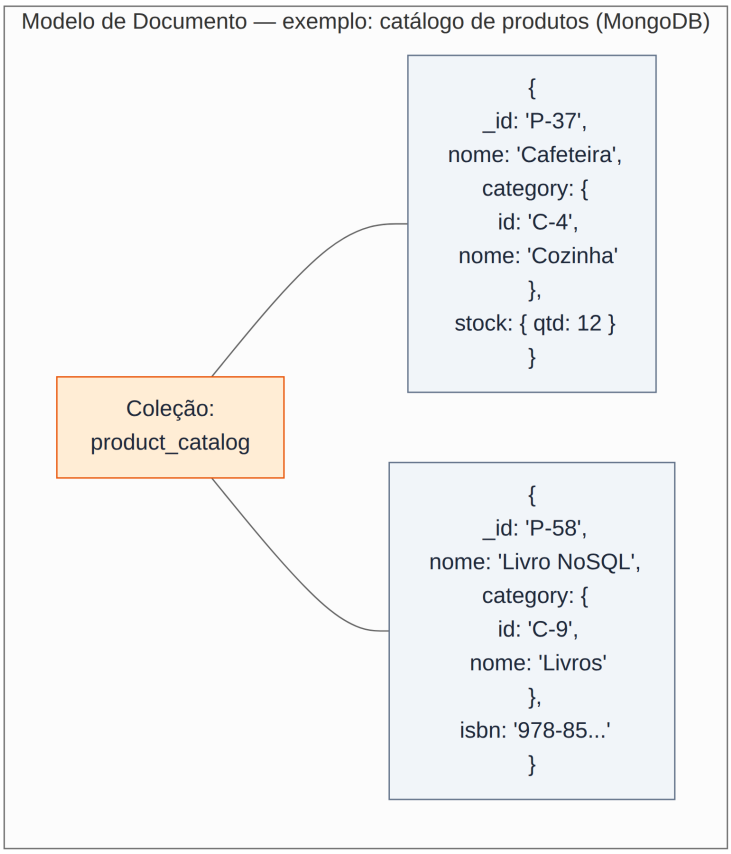


Figura 5 – Modelo de documento: documentos autocontidos com campos variáveis (MongoDB).
Fonte: Elaborado pelo autor (2026).

2.2.4 Grafo

Bancos de dados em grafo representam dados como **nós** (entidades) e **arestas** (relacionamentos tipados), com propriedades em ambos. Consultas exploram caminhos e vizinhanças de forma natural, o que é útil para recomendação, detecção de fraude e redes sociais. O Neo4j utiliza a linguagem Cypher para `MERGE`/`CREATE` de nós e relacionamentos.

Na ferramenta proposta, o destino grafo exige que o usuário declare explicitamente quais colunas do CSV identificam cada rótulo de nó e quais colunas alimentam propriedades do relacionamento (por exemplo, comprador $\rightarrow$ vendedor com atributos do pedido). A Figura 6 exemplifica um cenário de recomendação, em que nós `User` e `Product` são conectados por arestas tipadas (`PURCHASED`, `VIEWED`, `SIMILAR_TO`), algumas portando propriedades próprias.

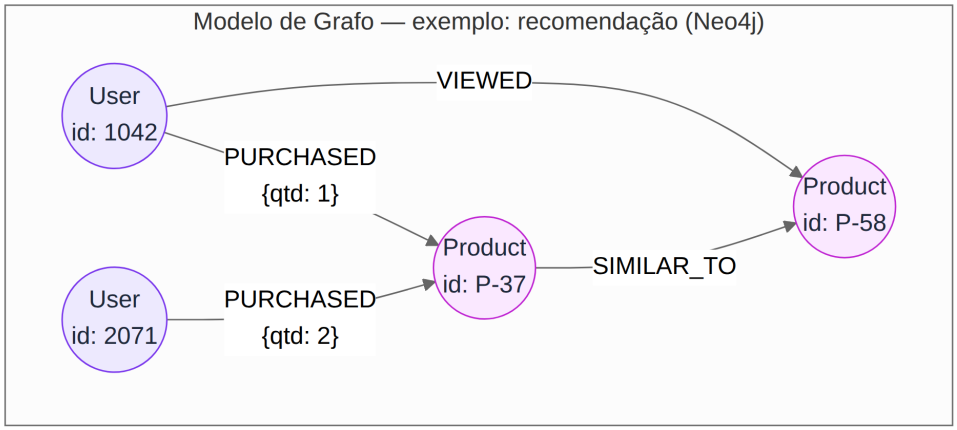


Figura 6 – Modelo de grafo: nós e arestas tipadas com propriedades (Neo4j).
Fonte: Elaborado pelo autor (2026).

2.3 BANCOS DE DADOS MULTIMODELO E ARQUITETURAS DE PERSISTÊNCIA POLIGLOTA

SGBDs multimodelo integram mais de um paradigma de dados no mesmo motor de gerência de dados (por exemplo, documento + chave-valor + grafo), oferecendo em geral uma interface de acesso unificada — como a linguagem AQL do SGBD ArangoDB, que combina filtros sobre estruturas de documento similares ao JSON e travessias em grafos em uma única abstração de modelo de dados que combina ambos os modelos NoSQL.

A persistência poliglota, por outro lado, combina vários SGBDs heterogêneos, cada um escolhido pela adequação a um caso de uso da aplicação (SADALAGE; FOWLER, 2013). Na literatura recente, essa ideia materializa-se como um SGPD (Sistema de Gerenciamento Poliglota de Dados): coordenação transparente entre tecnologias distintas, com requisitos de modelagem e expressividade de acesso (diferentes padrões de consulta) e capacidade de adaptação quando o *workload* evolui (KIEHN et al., 2022; GLAKE et al., 2022).

Tan et al. (2017) apresentam uma taxonomia útil para distinguir arquiteturas relacionadas: BD federado (fontes homogêneas, interface única), BD multistore (fontes heterogêneas, modelo canônico de consulta), BD *polystore* (fontes heterogêneas, múltiplas interfaces de acesso — exemplificadas por sistemas como o BigDAWG) e abordagens multimodelo (vários modelos no mesmo motor).

A escolha entre essas arquiteturas não é trivial. Roy-Hubara, Shoval e Sturm (2022) propõem critérios para selecionar quais modelos de dados usar em aplicações de persistência poliglota, destacando o *trade-off* entre especialização por SGBD e simplicidade operacional de um motor multimodelo.

O *Polyglot Import CSV* adota explicitamente a linha da persistência poliglota ao invés de multimodelo, conforme detalhado no capítulo 4. O usuário direciona entidades e filtros para PostgreSQL, Redis, MongoDB, Cassandra e Neo4j conforme o padrão de representação e acesso desejados.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta e discute ferramentas oferecidas na indústria por SGBDs e também propostas na literatura acadêmica relacionadas com a importação de dados por SGBDs heterogêneos. A preferência está em dados no formato CSV que devem ser importados por alguma tecnologia de banco de dados, que é o foco deste trabalho.

3.1 IMPORTAÇÃO DE ARQUIVO CSV PARA UM ÚNICO SGBD

Esta seção investiga mecanismos de importação de dados presentes em SGBDs populares na indústria que são considerados neste trabalho.

3.1.1 PostgreSQL

No SGBD PostgreSQL você utiliza o comando `COPY` para realizar importações de dados CSV (The PostgreSQL Global Development Group, 2026). É necessário especificar a tabela com os nomes das colunas após a cláusula `COPY`. A ordem das colunas deve ser a mesma que aquelas no arquivo CSV, conforme ilustra o exemplo a seguir:

```
COPY nome_da_tabela(col1, col2, col3)
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Caso o arquivo CSV contenha todas as colunas da tabela, você não precisa especificá-las explicitamente:

```
COPY nome_da_tabela
FROM 'C:/caminho_do_arquivo/dados_de_amostra.csv'
DELIMITER ','
CSV HEADER;
```

Você também deve inserir o caminho do arquivo CSV após a palavra-chave `FROM`. Como o formato do arquivo é CSV, você precisa especificar `DELIMITER`, bem como a cláusula `CSV`. A cláusula `HEADER` deve ser especificada para indicar que o arquivo CSV contém um cabeçalho. Quando o comando `COPY` importa dados, ele ignora o cabeçalho do arquivo.

O arquivo CSV deve ser lido diretamente pelo servidor PostgreSQL, não pela aplicação cliente. Portanto, ele deve ser acessível pela máquina do servidor PostgreSQL. Além disso, é necessário ter permissão de superusuário para executar com sucesso o comando `COPY`.

3.1.2 Redis

A maneira preferida de importar uma massa de dados para o Redis é gerar um arquivo de texto contendo o protocolo Redis, em formato bruto, para chamar os comandos necessários para inserir os dados requeridos (Redis Ltd., 2026).

Por exemplo, se é necessário gerar um grande conjunto de dados com bilhões de chaves no formato "chaveN → ValorN", deve ser criado um arquivo contendo os seguintes comandos:

```
SET Chave0 Valor0
SET Chave1 Valor1
SET ChaveN ValorN
```

Uma vez criado esse arquivo, a ação restante é alimentá-lo no Redis. No passado, a maneira de fazer isso era usar o netcat com o seguinte comando:

```
(cat dados.txt; sleep 10) | nc localhost 6379
```

Nas versões 2.6 ou posteriores do Redis, o utilitário `redis-cli` suporta um novo modo chamado modo *pipe* que foi projetado para realizar o carregamento em massa. O comando a ser executado neste caso é o seguinte:

```
cat dados.txt | redis-cli --pipe
```

O utilitário `redis-cli` imprime um pequeno resumo. A saída típica é semelhante a esta:

```
All data transferred. Waiting for the last reply...
Last reply received from server.
errors: 0, replies: 1000000
```

A linha de contagem ao final é a informação mais importante do resumo: `errors` indica quantos comandos foram rejeitados pelo servidor, e `replies` indica o total de respostas recebidas. Se `errors: 0`, todos os comandos foram processados com sucesso. Um valor maior que zero em `errors` significa que ao menos um comando falhou — seja por sintaxe inválida no arquivo, por tipo de dado incompatível com a chave existente, ou por qualquer outra razão que o Redis tenha rejeitado.

Vale notar que o `--pipe` não exibe cada resposta individualmente (como um `+OK` por `SET`), o que é intencional: imprimir bilhões de respostas destruiria o ganho de *performance* do modo *pipe* e inundaria o terminal. Em vez disso, o `redis-cli` lê e contabiliza todas as respostas silenciosamente conforme chegam, entregando apenas o agregado final.

Por fim, uma verificação útil: o valor de `replies` deve ser igual ao número de comandos enviados. Uma divergência entre os dois pode indicar que o arquivo de entrada estava truncado ou malformado, mesmo que `errors` apareça como zero.

3.1.3 MongoDB

No SGBD MongoDB, o processo de importação de dados em formato CSV é realizado através do utilitário `mongoimport` (MongoDB, Inc., 2026). Sua sintaxe básica exige que se informe o banco de dados de destino através do parâmetro `--db`, seguido do nome da coleção que receberá os dados, especificado em `--collection`. Caso esse último parâmetro seja omitido, o nome da coleção será automaticamente inferido a partir do nome do arquivo CSV fornecido.

```
mongoimport --db nome_do_db --collection nome_da_colecao \
  --type csv --headerline --ignoreBlanks \
  --file caminho/do/arquivo.csv
```

O tipo do arquivo de entrada deve ser declarado explicitamente através do parâmetro `--type`, que neste caso recebe o valor `csv`. Para que o `mongoimport` reconheça os nomes dos campos a partir do próprio arquivo, utiliza-se o parâmetro `--headerline`, que instrui o utilitário a tratar o conteúdo da primeira linha do CSV como o cabeçalho dos campos. Opcionalmente, o parâmetro `--ignoreBlanks` pode ser utilizado para que valores em branco presentes no arquivo sejam ignorados durante a importação, evitando que campos vazios sejam persistidos na coleção. Por fim, o caminho do arquivo CSV a ser importado é fornecido através do parâmetro `--file`.

3.1.4 Cassandra

A importação de dados CSV para o Cassandra utiliza o comando `sstableloader` e envolve vários passos (The Apache Software Foundation, 2026). Inicialmente, é necessário se certificar que o arquivo CSV está no formato adequado para o Cassandra. Isso geralmente envolve garantir que os tipos de dados e as colunas correspondem ao esquema da tabela no Cassandra.

O `sstableloader` requer que os dados estejam em um formato específico chamado SSTable. Para converter o CSV em SSTables, você pode usar o comando `cqlsh` fornecido pelo Cassandra. Um exemplo de uso é o seguinte:

```
cqlsh -e "COPY keyspace_name.table_name TO 'output_directory';"
```

Na sequência, é possível usar o `sstableloader` para carregar os dados. A sintaxe do comando é a seguinte:

```
sstableloader -d <hostname> -u <username> -pw <password> \
  output_directory/keyspace_name/table_name
```

- `<hostname>`: O endereço do nó Cassandra.
- `<username>`: O nome de usuário, se a autenticação estiver habilitada.
- `<password>`: A senha correspondente.

Este comando move os dados do diretório de saída para o Cassandra.

3.1.5 Neo4j

Usar `neo4j-admin` para importar grandes conjuntos de dados no Neo4j envolve alguns passos (Neo4j, Inc., 2026). Esta ferramenta de linha de comando é projetada para importações em massa eficientes.

Certifique-se de que seus arquivos CSV estejam estruturados corretamente e sigam o formato necessário para nós e relacionamentos. Crie um banco de dados não povoado:

```
neo4j-admin create-db --database=nome_do_seu_banco_de_dados \
  --from=diretorio_com_arquivos_csv
```

Substitua `nome_do_seu_banco_de_dados` pelo nome desejado para o novo banco de dados. A opção `--from` especifica o diretório onde estão localizados seus arquivos CSV. Execute o comando de importação com `neo4j-admin`. O comando exato depende do seu modelo de dados, mas pode se parecer com isto:

```
neo4j-admin import --database=nome_do_seu_banco_de_dados \
  --mode=csv \
  --nodes:Rotulo1 nos.csv \
  --nodes:Rotulo2 nos2.csv \
  --relationships:TIPO_RELACIONAMENTO relacionamentos.csv
```

Substitua *Rotulo1*, *Rotulo2*, *nos.csv*, *nos2.csv*, *TIPO_RELACIONAMENTO* e *relacionamentos.csv* pelos seus rótulos específicos e nomes de arquivo.

3.2 IMPORTAÇÃO DE ARQUIVO CSV PARA SGBD MULTIMODELO

Ferramentas comerciais e de código aberto tratam de **migração** ou **modelagem** entre tecnologias heterogêneas, embora raramente com o foco deste TCC (um único CSV largo + regras declarativas + cinco paradigmas).

- **ArangoDB** é um SGBD multimodelo nativo (documento/chave-valor/grafo) com `arangoimport` para CSV; a importação é para **um** motor multimodelo, não para cinco SGBDs independentes.
- **`mongoimport`** / **`COPY`** / **`redis-cli --pipe`** / **`cqlsh`** / **`neo4j-admin`** (seção 3.1) cobrem importação **para um único** produto por vez.
- **dbcrossbar** (DBCROSSBAR. . . , 2024) copia dados tabulares entre PostgreSQL, BigQuery, armazenamento em nuvem e CSV, incluindo conversão de esquemas; a ênfase é *pipeline* de migração, não mapeamento semântico de entidades aninhadas com filtros por valor.

- **Hackolade** (Hackolade, 2024) oferece *polyglot data modeling* com engenharia de esquema para dezenas de alvos; trata-se de **modelagem visual** e geração de artefatos, e não de um utilitário CLI único para importar um CSV operacional com validação prévia de filtros.

3.3 CONSIDERAÇÕES SOBRE OS TRABALHOS RELACIONADOS

A literatura sobre **metamodelos unificados** para SQL e NoSQL (por exemplo, U-Schema (CANDEL; RUIZ; GARCíA-MOLINA, 2022)) e sobre **evolução de esquema** em ambientes heterogêneos (CHILLóN et al., 2023; CHILLóN et al., 2024) informa decisões de validação e de representação de entidades no JSON de configuração, mas não substitui uma ferramenta de *bulk import* orientada a CSV.

Surveys recentes convergem em desafios abertos da gestão poliglota de dados: migração automatizada entre *stores* quando o *workload* muda, planejamento de consultas entre sistemas, gestão de esquemas multi-modelo e preservação das propriedades não funcionais de cada SGBD (KIEHN et al., 2022; GLAKE et al., 2022). Roy-Hubara, Shoval e Sturm (2022) tratam da **seleção de SGBDs** por fragmento de aplicação; Silva, Bornia e Mello (2024) mapeiam o estado da arte em **modelagem poliglota**. Nenhum desses trabalhos implementa, contudo, um utilitário declarativo que receba um CSV operacional único e o materialize simultaneamente em cinco paradigmas distintos com validação estática prévia.

Abordagens recentes reforçam essa tendência. Pereira, Guerra e Rosa (2025) propõem um *framework* de persistência poliglota **transparente**, com uma API unificada e extensível a diferentes tipos de banco, enquanto Sachdeva e Vasava (2024) exploram a combinação de bancos **multimodelo** e persistência poliglota para acomodar a variedade dos dados. Ambos os trabalhos atuam, porém, na **camada de acesso da aplicação** — oferecendo uma interface única para ler e gravar em múltiplos SGBDs em tempo de execução —, e não na **carga inicial declarativa** a partir de um CSV operacional, que é o foco da *Polyglot Import CSV*.

Sistemas *polystore* como o BigDAWG (DUGGAN et al., 2015) e *middlewares* de portabilidade em nuvem (KAUR; SHARMA; KAHLON, 2020) tratam de **consultas federadas** e movimentação de dados em ecossistemas complexos. Tan et al. (2017) classificam essas soluções (federado, multistore, polystore, poliglota) e avaliam transparência, autonomia e heterogeneidade — eixos nos quais importadores nativos (seção 3.1) e ferramentas de migração (seção 3.2) também divergem, mas sem convergir para um *pipeline* único de ingestão.

Ferramentas como **dbcrossbar** (DBCROSSBAR. . . , 2024) e **Hackolade** (Hackolade, 2024) aproximam-se do domínio de integração de dados, porém com foco

em *pipelines* de migração ou modelagem visual, respectivamente — não em regras declarativas sobre entidades aninhadas, filtros por valor (`each`) e validação por JSON Schema antes de qualquer conexão com SGBDs.

Lacuna identificada: a combinação de (i) um CSV largo como fonte operacional, (ii) configuração JSON validada estaticamente, (iii) materialização distinta por paradigma (relacional, chave-valor, documento, colunar, grafo) e (iv) modo *dry-run* para revisão prévia. O *Polyglot Import CSV* ocupa essa lacuna como ferramenta de **bootstrap de persistência poliglota** — carga inicial e reprodutível de dados — enquanto *middlewares* e *polystores* assumem dados já residentes ou consultas em tempo de execução. Trabalhos futuros podem integrar a ferramenta a mecanismos de migração adaptativa descritos por Kiehn et al. (2022) e Glake et al. (2022).

4 A PROPOSTA DA FERRAMENTA POLYGLOTIMPORTCSV

4.1 VISÃO GERAL

A ferramenta **PolyglotImportCSV** (pacote Python `polyglot-import-csv`) importa dados em formato CSV para vários SGBDs em uma única execução. Conforme ilustra a Figura 7, a ferramenta recebe como entrada os **arquivos de dados** (um ou mais CSV) e **dois arquivos de configuração** em formato JSON — o arquivo de **importação**, que declara quais CSVs alimentam a carga e mapeia as suas colunas para as entidades e relacionamentos desejados em cada SGBD, e o arquivo de **conexão**, que declara quais SGBDs estão disponíveis e como alcançá-los. O processamento é disparado por um comando de linha de comando (CLI), que primeiro **valida** as entradas — cada arquivo de configuração é conferido contra um **JSON Schema** embutido e, em seguida, contra os próprios CSVs — e só então executa a importação. A **saída** são os dados persistidos em cada SGBD presente na configuração, dentre os suportados: PostgreSQL, MongoDB, Cassandra, Redis e Neo4j.

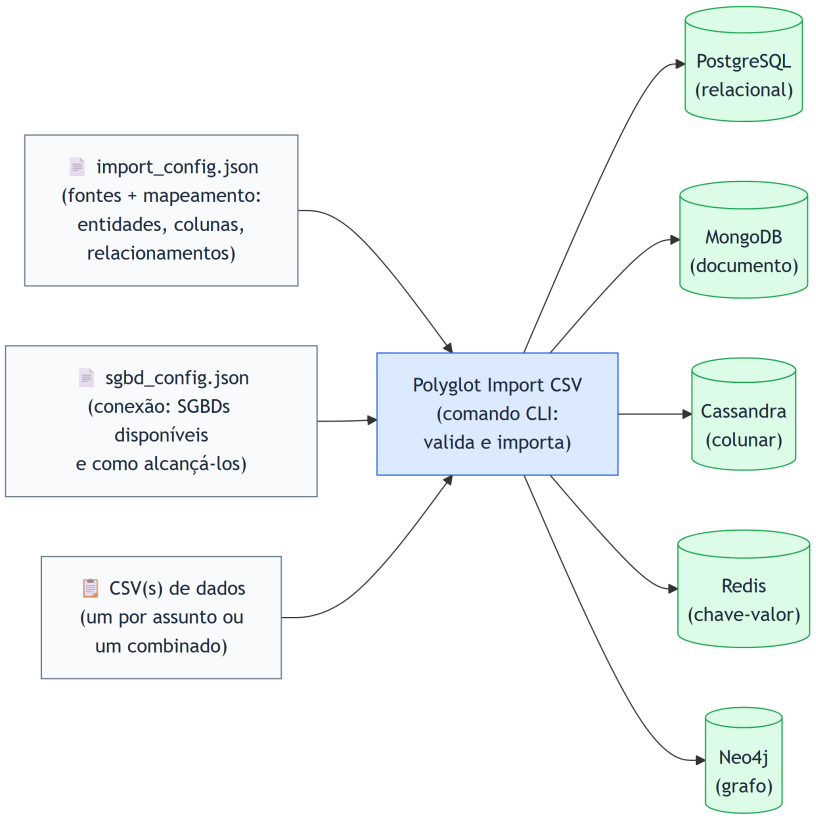


Figura 7 – Visão geral da ferramenta: entradas, processamento e saídas.
Fonte: Elaborado pelo autor (2026).

Diferentemente de uma importação amarrada a um único arquivo, os dados de

entrada são declarados em um bloco `sources` dentro do arquivo de importação, o que habilita **dois modos de uso**: no modo **multifonte**, cada entidade lê de um CSV próprio (um arquivo por assunto); no modo **combinado**, um único CSV largo carrega todos os assuntos, distinguidos por uma coluna de origem. Ambos os modos e o formato completo dos dois arquivos de configuração são detalhados na seção 4.2; as validações realizadas antes da importação, na seção 4.2.9; o algoritmo de importação em si, na seção 4.3; e a camada de observabilidade e métricas, na seção 4.4. O comando principal da CLI é o seguinte:

o que é um assunto??

```
python -m polyglotimportcsv \
  --config caminho/import_config.json \
  --sgbd-config caminho/sgbd_config.json \
  [--source NOME=CAMINHO] \
  [--dry-run] \
  [--create-schema / --no-create-schema] \
  [--only postgres,redis] \
  [--log-level INFO] \
  [--show-data / --no-data] \
  [--benchmark]
```

Cada parte do comando tem o seguinte papel. A opção `--config` é **obrigatória** e indica o caminho do arquivo JSON de importação, que já contém, em seu bloco `sources`, os caminhos dos CSVs a carregar — por isso não há mais um argumento posicional de arquivo de dados. A opção `--sgbd-config` indica o caminho do arquivo JSON de conexão dos SGBDs; se omitida, a ferramenta procura um arquivo `sgbd_config.json` no mesmo diretório do arquivo de importação. A opção `--source NOME=CAMINHO` **sobrescreve**, em tempo de execução, o caminho de uma fonte declarada no bloco `sources` sem editar o arquivo de configuração (por exemplo, para apontar a mesma configuração a um conjunto de dados maior); pode ser repetida uma vez por fonte.

como assim? na versão anterior do comando havia um argumento posicional? Será que precisa dizer isso?

se essa opção pode ser omitida, então ela é opcional no comando, correto?

As demais opções ~~são facultativas e~~ agrupam-se em dois conjuntos. O primeiro controla **o que** e **onde** importar: `--dry-run` executa somente as validações e a preparação dos dados e reporta quantos registros **seriam** importados em cada destino, sem conectar a nenhum SGBD, o que é útil para revisar o mapeamento antes da carga real; `--create-schema` e `--no-create-schema` controlam se a ferramenta cria as estruturas de armazenamento necessárias (tabelas no PostgreSQL, *keyspace* e tabelas no Cassandra) antes de gravar os dados, sendo a criação ativada por padrão; e `--only` restringe a execução a um subconjunto dos SGBDs configurados (por exemplo, `--only postgres,redis`). O segundo conjunto governa a **observabilidade** da execução — `--log-level` ajusta o nível de detalhe das mensagens no terminal, `--show-data`/`--no-data` força ou suprime a exibição dos dados materializados e `--benchmark` registra métricas de desempenho por fase —; essas três opções são descritas em detalhe na seção 4.4.

4.2 FORMATO DE CONFIGURAÇÃO (JSON SCHEMA)

A configuração é dividida em **dois arquivos JSON**, cada um validado estaticamente por um **JSON Schema** próprio embutido em `src/polyglotimportcsv/schemas/`: o `import_config.json` define as **fontes de dados** e o **mapeamento** de entidades, relacionamentos e colunas para cada SGBD; o `sgbd_config.json` define a **configuração de conexão** de cada SGBD, isto é, quais bancos estão disponíveis e como alcançá-los.

onde está esse repositório? Mostrar um link para o leitor poder acessar…

Essa separação atende a duas preocupações distintas: o *mapeamento* muda conforme o domínio dos dados; a *conexão* muda conforme o ambiente (desenvolvimento, testes, produção). Um SGBD só pode ser referenciado no arquivo de importação se estiver **declarado** no arquivo de conexão — esta e as demais regras de validação são detalhadas na seção 4.2.9. A representação completa de ambos os schemas na forma serializada JSON Schema 2020-12 encontra-se no Apêndice A.

4.2.1 Fontes de Dados e os Dois Modos de Importação

esse bloco “sources” o leitor não entende o que é aqui neste momento, por quê a estrutura dos arquivos de configuração não foi explicados. Eu acho que a essa seção deveria vir depois da explicação da estrutura dos arquivos. A estrutura dos arquivos deve ser a 1a coisa explicada

O bloco `sources`, obrigatório no arquivo de importação, nomeia as fontes de dados e determina o modo de importação. Cada entrada associa um **nome de fonte** a um arquivo CSV, em uma de duas formas.

Na primeira, **multifonte**, o valor é diretamente o caminho de um CSV, e cada fonte corresponde a um assunto do domínio, com o seu próprio arquivo. A Listagem 4.1 mostra o bloco `sources` do cenário de e-commerce nesse modo: quatro fontes, uma por tipo de evento da loja.

que cenário de e-commerce é esse? Vc já explicou éle antes? Se não explicou o leitor não vai saber…

```
"sources": {
  "stock":          "ecommerce_stock.csv",
  "purchase":       "ecommerce_purchase.csv",
  "select_product": "ecommerce_select_product.csv",
  "add_to_cart":    "ecommerce_add_to_cart.csv"
}
```

Listagem 4.1 – Bloco `sources` no modo multifonte (um CSV por assunto).

Na segunda, **combinada**, o valor é um objeto `{ "file": ..., "origin_column": true }`, indicando que um **único** CSV largo reúne todos os assuntos, sendo a sua **primeira coluna** o rótulo de origem de cada linha. A Listagem 4.2 mostra o mesmo cenário nesse modo: uma só fonte, apontando para o arquivo com a coluna de origem.

```
"sources": {
  "ecommerce": { "file": "ecommerce_join.csv", "origin_column": true }
}
```

Listagem 4.2 – Bloco `sources` no modo combinado (um CSV com coluna de origem).

Ao carregar uma fonte combinada, a ferramenta lê a primeira coluna como a **origem**, remove-a das colunas de dados e registra, além da fonte inteira, uma fonte

te a pergunta: já foi apresentado esse domínio de e-commerce antes? Senão o leitor não entenderá o que são esses termos…

derivada para cada valor distinto de origem. Dessa forma, os nomes `stock`, `purchase`, `select_product` e `add_to_cart` tornam-se fontes disponíveis exatamente como no modo multifonte, e o restante da configuração — o mapeamento de cada SGBD — permanece **idêntico** entre os dois modos. Os dois arquivos de importação que acompanham o repositório são, de fato, byte a byte iguais exceto pelo bloco `sources`.

Em qualquer dos modos, cada linha carrega uma **pseudo-coluna** `_source` com a sua origem: o nome da fonte, no modo multifonte, ou o valor da coluna de origem, no modo combinado. Essa pseudo-coluna pode ser mapeada como qualquer outra coluna — no cenário de e-commerce, ela alimenta o tipo de evento na tabela do Cassandra, como se verá na seção 4.3.2.

Vínculo entre entidade e fonte. Cada entidade declara de qual fonte lê por meio do campo `source`, que admite três formas: (i) uma **string** com o nome de uma fonte (`"source": "stock"`); (ii) uma **lista** de nomes (`"source": ["stock", "purchase"]`), caso em que as fontes são **unidas** em uma só tabela — colunas ausentes em uma das partes ficam vazias e a pseudo-coluna `_source` preserva a origem de cada linha; ou (iii) a **omissão** do campo, caso em que a entidade se liga à fonte que tiver o seu próprio nome. Essa flexibilidade permite que entidades diferentes leiam de fontes diferentes na mesma execução, sem depender de uma coluna discriminadora presente em todas as linhas.

4.2.2 Visão Geral da Estrutura dos Arquivos de Configuração

Antes de detalhar cada SGBD de destino suportado — identificado nos arquivos de configuração pelos blocos `postgres`, `mongodb`, `cassandra`, `redis` e `neo4j` — esta subseção apresenta a estrutura dos dois schemas como um todo, fornecendo um mapa mental que orienta a leitura das subseções seguintes. A Figura 8 sintetiza graficamente o schema do arquivo de importação (`import_config`) e a Figura 9, o schema do arquivo de conexão (`sgbd_config`).

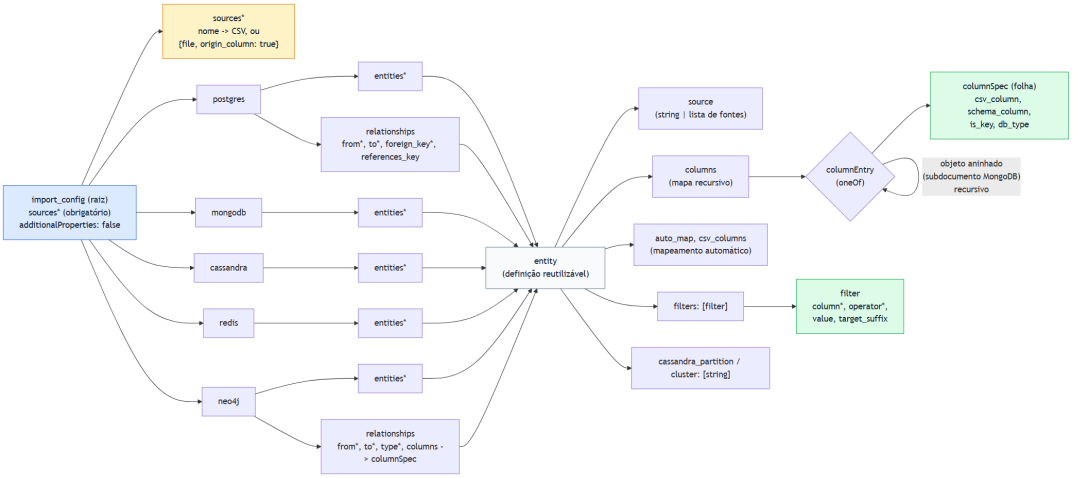


Figura 8 – Estrutura do JSON Schema de importação (`import_config`).
Fonte: Elaborado pelo autor (2026).

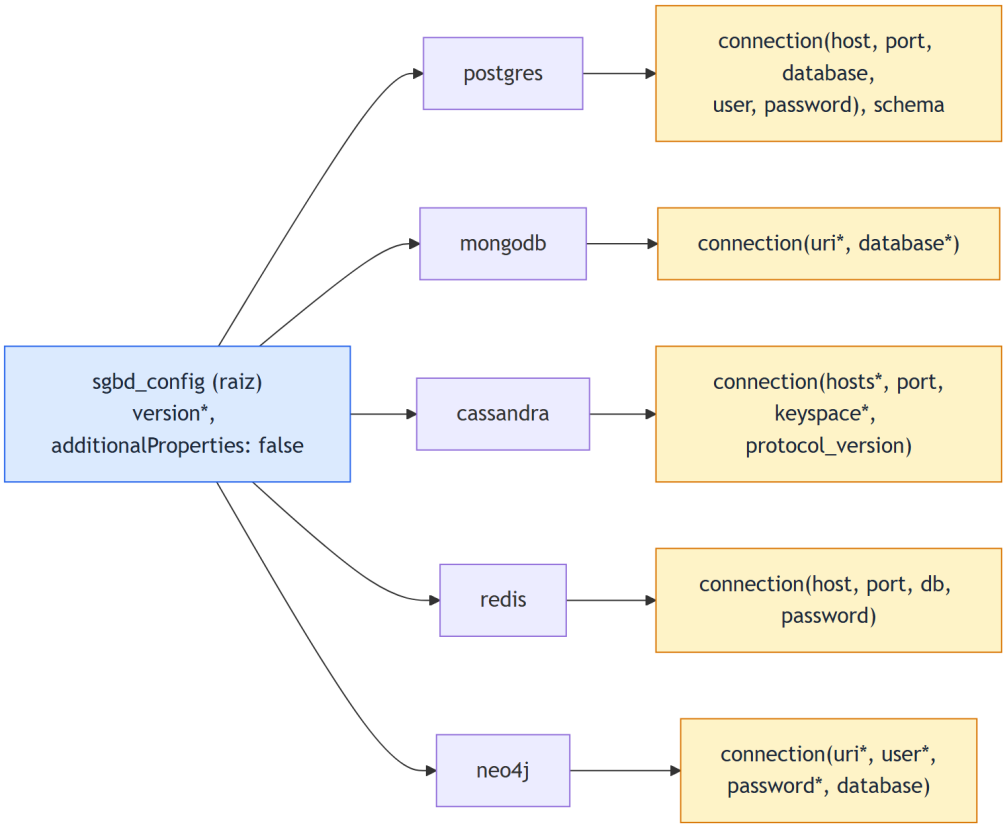


Figura 9 – Estrutura do JSON Schema de conexão (`sgbd_config`).
Fonte: Elaborado pelo autor (2026).

Dizer aqui que as diferentes partes dos esquemas são explicadas a seguir (introduzir os próximos parágrafos…)

Raízes por SGBD. Ambos os schemas partilham a mesma organização: um bloco opcional por SGBD (`postgres`, `mongodb`, `cassandra`, `redis`, `neo4j`), com `additionalProperties: false` rejeitando chaves desconhecidas. A diferença está no conteúdo de cada bloco e na raiz: o arquivo de importação exige o bloco `sources` (seção 4.2.1) e descreve *o que* mapear; o de conexão exige um campo `version` e

sugiro trazer para cá
essa explicação do bloco
sources

descreve *onde* gravar.

O arquivo de importação e a definição reutilizável `entity`. Seguindo as setas a partir de `import_config`, cada bloco de SGBD expande-se em um mapa `entities` (e, para PostgreSQL e Neo4j, também em `relationships`). O ponto central da Figura 8 — e da economia de vocabulário do schema — é que **todas as entidades, de todos os SGBDs, convergem para uma única definição reutilizável**, rotulada `entity`. Em vez de cinco formatos distintos, há apenas um: toda entidade possui um mapa `columns` (o mapeamento de colunas) e campos opcionais como `source` (o vínculo com a fonte, seção 4.2.1), `auto_map` e `csv_columns` (mapeamento automático, seção 4.2.3), `filters` (predicados sobre as linhas) e os campos específicos do Cassandra (`cassandra_partition` e `cassandra_cluster`).

O mapeamento recursivo de colunas. À direita da Figura 8, `columns` remete a `columnEntry`, definido como uma escolha (`oneOf`) entre duas formas: (i) uma **folha** — o `columnSpec`, que liga uma coluna do CSV a um atributo do destino por meio dos campos `csv_column`, `schema_column`, `is_key` e `db_type`; ou (ii) um **objeto aninhado**, que contém novos `columnEntry`. Essa recursão é o que permite, com um único mecanismo, tanto o mapeamento plano (PostgreSQL, Cassandra, Redis) quanto os subdocumentos do MongoDB. As arestas tipadas do Neo4j e as chaves estrangeiras do PostgreSQL aparecem em `relationships`, reutilizando o mesmo `columnSpec` para eventuais propriedades.

O arquivo de conexão. Na Figura 9, cada bloco de SGBD de `sgbd_config` carrega apenas um descritor `connection` com os parâmetros de acesso pertinentes àquela tecnologia (por exemplo, `uri` para MongoDB e Neo4j; `hosts` e `keyspace` para Cassandra). É uma estrutura deliberadamente rasa: indica *onde* alcançar cada SGBD, sem nenhuma informação de mapeamento.

As subseções seguintes percorrem essa estrutura na ordem em que as duas figuras são lidas: primeiro a raiz e os elementos comuns a todos os SGBDs (`columnSpec`, `entity`, mapeamento automático e filtros) e, em seguida, as particularidades de cada SGBD, ilustradas com fragmentos concretos de configuração.

4.2.3 Raiz e Elementos Comuns

Como antecipado na visão geral, ambos os arquivos organizam-se em um bloco opcional por SGBD. A cláusula `additionalProperties: false` **rejeita** qualquer propriedade não prevista, impedindo que erros de digitação como `postgress` passem despercebidos.

Antes de começar a mostrar esses itens de explicação tens que fazer uma introdução, como já comentei antes

columnSpec. Cada **folha** do mapa `columns` descreve como um valor do CSV alimenta um atributo no destino. Quatro campos opcionais compõem esse mapeamento, conforme a Tabela 1.

não tá muito didática essa explicação. Pq vc não vai explica cada arquivo de configuração separadamente, explicando as coisas na ordem em que aparecem no esquema JSON: 1o a parte da raiz, depois a parte sources, as partes de cada SGBD, e assim por diante. Ficaria mais fácil para o leitor ir acompanhando. Aqui vc já começa explicando o columnSpec, que é um componente lá do final da estrutura… Fica confuso assim… Ainda, para cada parte explicada já mostrar um exemplo em JSON para facilitar o entendimento

Tabela 1 – Campos de mapeamento de coluna (`columnSpec`).

Campo	Tipo	Obrigatório	Descrição
`csv_column`	*string* ou inteiro (≥ 1)	não	Cabeçalho CSV ou índice da coluna de dados (base 1). Padrão: chave JSON.
`schema_column`	*string*	não	Nome do atributo no SGBD destino. Padrão: chave JSON.
`is_key`	booleano	não	Chave primária, chave de `MERGE` (Neo4j) ou chave Redis.
`db_type`	*string*	não	Tipo SQL/CQL para geração de DDL.

Fonte: Elaborado pelo autor (2026).

Difícil de entender a finalidade desse exemplo sem explicação do esquema JSON. É muita explicação teórica da estrutura do esquema JSON sem exemplificação. Sugiro uma reorganização geral dessas explicações do esquema JSON de configuração, cf sugerido no comentário anterior… Explicar a partir de um exemplo eu acho que ajuda mais, fica mais didático.

A chave JSON de cada entrada em `columns` identifica o mapeamento e serve de **padrão** tanto para a coluna de origem no CSV quanto para o atributo de destino no SGBD; por isso a forma mínima (`"campo": {}`) basta quando os três nomes coincidem. Quando divergem, `csv_column` redefine a origem — por nome de cabeçalho ou por índice posicional de base 1, útil para CSVs sem cabeçalho — e `schema_column` redefine o destino no SGBD. Por ser um identificador independente da origem, a chave permite ainda que uma mesma coluna do CSV alimente atributos distintos (entradas com `csv_column` repetido), recurso pontual para cenários de denormalização.

o que queres dizer com "coincidir" ou "divergir" ?? O que pode divergir?? Deixar mais claro…

Mapeamento automático (`auto_map` e `csv_columns`). Nem sempre é prático enumerar coluna a coluna. Quando o campo `columns` é omitido, ou quando `auto_map` é `true`, a entidade **mapeia automaticamente** todas as colunas da sua fonte, atribuindo a cada uma o nome e um tipo de destino inferido do próprio dado (adiante, em **Inferência de tipos**); entradas manuais em `columns` sobrepõem-se às automáticas coluna a coluna, permitindo, por exemplo, declarar apenas a chave e deixar o restante ser inferido. O campo `csv_columns` — válido apenas com mapeamento automático — **restringe** o conjunto auto-mapeado a colunas específicas, indicadas por nome, por índice de base 1 ou por intervalos como `"1-5"`.

o campo columns pode ser omitido? Mas ele não é sempre importante para informar como cada coluna do CSV será mapeada? Não está claro para mim isso. Acho que logo no início precisas deixar claro o que é obrigatório e o que opcional do esquema JSON de cada arquivo.

Inferência de tipos. Todos os valores são lidos inicialmente como texto, preservando o conteúdo original do arquivo. Em seguida, a ferramenta classifica cada coluna segundo o tipo predominante entre os seus valores — inteiro, real, data/hora, booleano, texto ou vazia — e converte os valores para o tipo nativo correspondente. Essa classificação também define o tipo de destino (`db_type`) usado na geração de DDL quando a coluna é auto-mapeada, segundo a correspondência da Tabela 2.

Tabela 2 – Correspondência entre tipo inferido e tipo de destino na geração de DDL.

Tipo inferido	**Tipo de destino (`db_type`)**
inteiro (*integer*)	`BIGINT`
real (*float*)	`NUMERIC`
data/hora (*datetime*)	`TIMESTAMPTZ`
booleano (*boolean*)	`BOOLEAN`
texto (*string*) ou vazia	`TEXT`

Fonte: Elaborado pelo autor (2026).

entity. Uma entidade representa um alvo lógico: tabela (PostgreSQL/Cassandra), coleção (MongoDB), rótulo de nó (Neo4j) ou entidade Redis. O único campo estritamente necessário é o vínculo com a fonte (explícito em `source` ou implícito pelo nome); `columns` pode ser omitido em favor do mapeamento automático. São opcionais `auto_map` e `csv_columns` (mapeamento automático), `filters` (predicados sobre as linhas), e `cassandra_partition` e `cassandra_cluster` (apenas Cassandra).

Filtros. Além do vínculo com a fonte, uma entidade pode declarar `filters`: predicados opcionais que **restringem** as linhas participantes. Cada item exige `column` e `operator` (`==`, `!=`, `>`, `<`, `>=`, `<=`, `in`, `not_in`, `each`). O operador `each` particiona a entidade por valores distintos da coluna, gerando múltiplos alvos com sufixo opcional. Diferentemente da seleção da origem — feita pelo bloco `sources` e pelo campo `source` —, os filtros são um refinamento adicional; no cenário de e-commerce, em que cada assunto já vem de uma fonte própria, eles não são necessários e não aparecem na configuração.

4.2.4 PostgreSQL

Arquivo de importação. O bloco `postgres` contém `entities` (entidades planas — aninhamento de `columns` é rejeitado) e, opcionalmente, `relationships` para declarar chaves estrangeiras. O campo `db_type` em cada `columnSpec` alimenta a geração de DDL (`CREATE TABLE`) quando `--create-schema` está ativo. A Listagem 4.3 mostra um fragmento da importação no domínio de e-commerce.

```
"postgres": {
  "entities": {
    "inventory": {
      "source": "stock",
      "columns": {
        "product_id":         { "is_key": true, "db_type": "BIGINT" },
        "quantity_available": { "db_type": "BIGINT" },
        "last_restock_date":  { "db_type": "TIMESTAMPTZ" },
        "price":              { "db_type": "NUMERIC(14,2)" }
```

```
      }
    },
    ...
  },
  "relationships": {
    "product_category": {
      "from": "products",
      "to":   "categories",
      "foreign_key":    "category_id",
      "references_key": "category_id"
    },
    ...
  }
}
```

Listagem 4.3 – Fragmento de importação PostgreSQL (e-commerce).

As reticências indicam que o arquivo real declara outras três entidades (`categories`, `products` e `orders`) e uma segunda chave estrangeira (`order_product`), omitidas aqui por brevidade; o arquivo completo é reproduzido no Apêndice C. O campo `source` liga a entidade `inventory` à fonte `stock` (as linhas de reposição de estoque), das quais projeta quatro colunas, com `product_id` como chave.

Arquivo de conexão. O bloco `postgres` contém `connection` (host, port, database, user, password) e o campo `schema` (padrão `public`), que define o schema PostgreSQL de destino das tabelas. A Listagem 4.4 mostra um exemplo de configuração de conexão.

```
"postgres": {
  "connection": {
    "host": "127.0.0.1", "port": 5432,
    "database": "ecommerce",
    "user": "postgres", "password": "postgres"
  },
  "schema": "public"
}
```

Listagem 4.4 – Fragmento de conexão PostgreSQL.

4.2.5 MongoDB

Arquivo de importação. O bloco `mongodb` contém apenas `entities`. O diferencial do MongoDB é o suporte a **aninhamento de `columns`**: chaves JSON aninhadas produzem subdocumentos BSON, sem necessidade de nenhum campo adicional — o próprio mapa recursivo expressa a hierarquia. A Listagem 4.5 exemplifica esse aninhamento: as chaves `category` e `stock` não são colunas do CSV, e sim objetos que agrupam outras colunas, produzindo dois subdocumentos dentro de cada documento `product_catalog`, também alimentado pela fonte `stock`.

```
"mongodb": {
  "entities": {
    "product_catalog": {
      "source": "stock",
      "columns": {
        "product_id":   {},
        "product_name": {},
        ...
        "category": {
          "category_id": {},
          "category_name": {}
        },
        "stock": {
          "quantity_available": {},
          "last_restock_date": {}
        }
      }
    }
  }
}
```

Listagem 4.5 – Fragmento de importação MongoDB com subdocumentos.

As reticências omitem as demais colunas cadastrais do produto (variante, marca, descrição, imagem e preço), mapeadas de forma direta como `product_id` e `product_name`; o bloco completo encontra-se no Apêndice C.

Arquivo de conexão. O bloco `mongodb` contém `connection` com `uri` (string de conexão MongoDB, incluindo usuário e senha quando necessário) e `database` (nome do banco de destino), como na Listagem 4.6.

```
"mongodb": {
  "connection": {
    "uri": "mongodb://127.0.0.1:27017",
    "database": "ecommerce"
  }
}
```

Listagem 4.6 – Fragmento de conexão MongoDB.

4.2.6 Apache Cassandra

Arquivo de importação. O bloco `cassandra` contém `entities` planas. Dois campos específicos controlam a chave primária composta do Cassandra: `cassandra_partition` (lista de colunas que formam a chave de partição) e `cassandra_cluster` (colunas de agrupamento). O campo `cassandra_partition` é obrigatório para gerar o schema Cassandra: sua ausência só é detectada quando `--create-schema` está ativo, pois o CQL exige ao menos uma coluna de partição na cláusula `PRIMARY KEY`; esse campo é independente de `is_key`, que no Cassandra não tem papel na definição da

chave primária. A Listagem 4.7 ilustra uma entidade que **une** todas as fontes e usa uma chave composta.

```
"cassandra": {
  "entities": {
    "user_activity_log": {
      "source": ["stock", "purchase", "select_product", "add_to_cart"],
      "columns": {
        "user_id":             {},
        "timestamp":           { "schema_column": "event_time" },
        "_source":             { "schema_column": "event_type" },
        "product_id":          {},
        "order_number":        {},
        "selected_product_id": {},
        "shopping_cart_id":    {}
      },
      "cassandra_partition": ["user_id"],
      "cassandra_cluster":   ["timestamp"]
    }
  }
}
```

Listagem 4.7 – Fragmento de importação Cassandra com união de fontes e chave composta.

A entidade `user_activity_log` declara `source` como a **lista** das quatro fontes: as suas linhas são unidas em uma única tabela de eventos. A pseudo-coluna `_source` — que registra a origem de cada linha — é mapeada para a coluna `event_type`, distinguindo os tipos de evento na tabela CQL; de modo análogo, `schema_column` renomeia `timestamp` para `event_time`. É um exemplo concreto de como `csv_column`/`_source` e `schema_column` atuam independentemente.

Arquivo de conexão. O bloco `cassandra` contém `connection` com `hosts` (lista de endereços de nós), `port` (padrão 9042), `keyspace` e `protocol_version` opcional, conforme a Listagem 4.8.

```
"cassandra": {
  "connection": {
    "hosts": ["127.0.0.1"],
    "port": 9042,
    "keyspace": "ecommerce"
  }
}
```

Listagem 4.8 – Fragmento de conexão Cassandra.

4.2.7 Redis

Arquivo de importação. O bloco `redis` contém `entities`. A restrição fundamental é que **cada entidade deve ter exatamente uma coluna marcada com `is_key`**: esse valor se torna a chave Redis — usada exatamente como aparece no CSV — e todos os demais campos mapeados compõem o valor JSON armazenado. A Listagem 4.9

mostra as duas entidades do cenário e-commerce e ilustra o mapeamento automático: `shopping_cart` declara apenas a chave e usa `auto_map` para mapear todas as demais colunas da fonte `add_to_cart`.

```
"redis": {
  "entities": {
    "shopping_cart": {
      "source": "add_to_cart",
      "auto_map": true,
      "columns": {
        "shopping_cart_id": { "is_key": true }
      }
    },
    "user_session": {
      "source": "select_product",
      "columns": {
        "user_id":    { "is_key": true },
        "user_name":  {},
        "user_email": {},
        "timestamp":  { "schema_column": "last_seen" }
      }
    }
  }
}
```

Listagem 4.9 – Fragmento de importação Redis (e-commerce), com mapeamento automático.

A entidade `shopping_cart` lê da fonte `add_to_cart` e usa como chave o identificador do carrinho (`shopping_cart_id`); com `auto_map`, o valor armazenado reúne automaticamente as demais colunas da fonte. A entidade `user_session` lê da fonte `select_product`, usa como chave o identificador do usuário e renomeia `timestamp` para `last_seen`, registrando a última atividade de navegação de cada usuário.

Arquivo de conexão. O bloco `redis` contém `connection` com `host`, `port` (padrão 6379), `db` (índice do banco, padrão 0) e `password` opcional, como na Listagem 4.10.

```
"redis": {
  "connection": {
    "host": "127.0.0.1",
    "port": 6379,
    "db": 0
  }
}
```

Listagem 4.10 – Fragmento de conexão Redis.

4.2.8 Neo4j

Arquivo de importação. O bloco `neo4j` contém `entities` (nós) e `relationships` (arestas tipadas). Cada entidade declara um rótulo (*label*) de nó e deve ter exatamente uma coluna `is_key`, usada como predicado de identificação no `MERGE` Cypher. As arestas

em `relationships` declaram `from` (rótulo de origem), `to` (rótulo de destino), `type` (tipo da aresta em maiúsculas) e um mapa `columns` opcional com propriedades da aresta. A Listagem 4.11 mostra um nó de cada rótulo, ligados por uma aresta.

```
"neo4j": {
  "entities": {
    "User": {
      "source": "purchase",
      "columns": {
        "user_id":    { "is_key": true },
        "user_name":  {},
        "user_email": {}
      }
    },
    "Product": {
      "source": "stock",
      "columns": {
        "product_id":    { "is_key": true },
        "product_name":  {},
        "product_brand": {}
      }
    }
  },
  "relationships": {
    "PURCHASED": {
      "from": "User",
      "to":   "Product",
      "type": "PURCHASED",
      "columns": {
        "order_number": { "is_key": true },
        "quantity":     {},
        "price":        {},
        "rating":       {}
      }
    }
  }
}
```

Listagem 4.11 – Fragmento de importação Neo4j (nós e aresta).

O nó `User` lê da fonte `purchase` (os compradores) e o nó `Product`, da fonte `stock` (os produtos repostos); a aresta `PURCHASED` liga um ao outro pela fonte de origem (`purchase`), decorada com quantidade, preço e nota da avaliação.

Arquivo de conexão. O bloco `neo4j` contém `connection` com `uri` (bolt://...), `user`, `password` e `database` opcional (padrão `neo4j`), conforme a Listagem 4.12.

```
"neo4j": {
  "connection": {
    "uri":      "bolt://127.0.0.1:7687",
    "user":     "neo4j",
    "password": "password",
    "database": "neo4j"
  }
}
```

Listagem 4.12 – Fragmento de conexão Neo4j.

4.2.9 Validação, Consistência entre Arquivos e Fluxo de Carregamento

A validação ocorre em **três camadas** antes de qualquer conexão com os SGBDs. A primeira é a validação por **JSON Schema**: cada arquivo é validado contra o seu schema (sintaxe, tipos e propriedades permitidas) — `import_config.schema.json` para o arquivo de importação e `sgbd_config.schema.json` para o arquivo de conexão. A segunda é a **consistência entre arquivos** (`config_parser.merge_configs`): todo SGBD referenciado no arquivo de importação deve estar declarado no arquivo de conexão. A terceira é a **validação cruzada com os dados**: ao carregar as fontes (`sources.load_sources`) e vincular cada entidade à sua fonte (`mapping_resolver`), confere-se se as colunas referenciadas existem, se os SGBDs de modelo plano não usam `columns` aninhados, se as colunas de partição e agrupamento do Cassandra existem, se os filtros são coerentes e se as chaves estrangeiras do PostgreSQL e as arestas do Neo4j referenciam entidades válidas (`validation.validate_backend_entities`).

Cada camada levanta uma exceção específica da hierarquia `BusinessException`, o que torna o diagnóstico preciso: `ConfigError` para erros de schema ou de consistência entre os arquivos; `SourceError` para problemas com os arquivos de fonte (por exemplo, um CSV inexistente ou uma coluna de origem vazia no modo combinado); `MappingError` para incoerências entre o mapeamento e os dados; e `ImportExecutionError` para falhas na conexão ou na gravação em um SGBD. Qualquer falha nas três primeiras camadas **impede** o início da importação. A Figura 10 apresenta o fluxo completo de carregamento, com os pontos de falha de cada camada e a convergência dos dois arquivos em uma configuração única, à qual os dados das fontes se juntam apenas na validação cruzada (camada 3). O diagrama evidencia por que a separação em dois arquivos isola falhas: um `sgbd_config.json` inválido é detectado independentemente do `import_config.json`, sem necessidade de ler os CSVs.

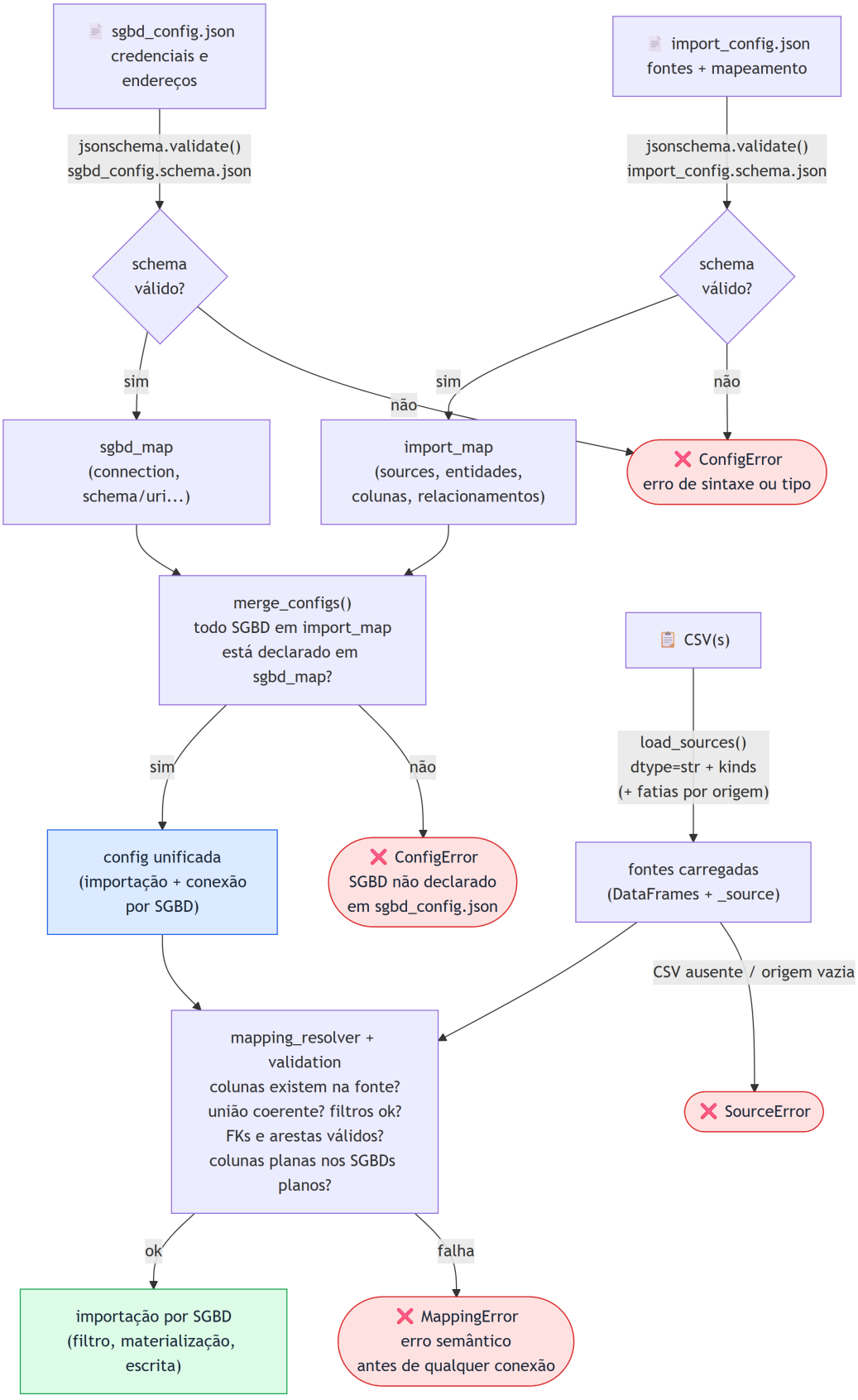


Figura 10 – Fluxo de carregamento e validação das entradas da ferramenta.

Fonte: Elaborado pelo autor (2026).

4.3 ALGORITMO DE EXECUÇÃO DA IMPORTAÇÃO

Definidos o formato de configuração e as validações, esta seção descreve como a ferramenta executa a importação propriamente dita, desde o comando de linha de comando apresentado na seção 4.1 até a persistência em cada SGBD. A Figura 11 apresenta o fluxo completo de execução; cada etapa do diagrama é explicada textualmente na sequência.

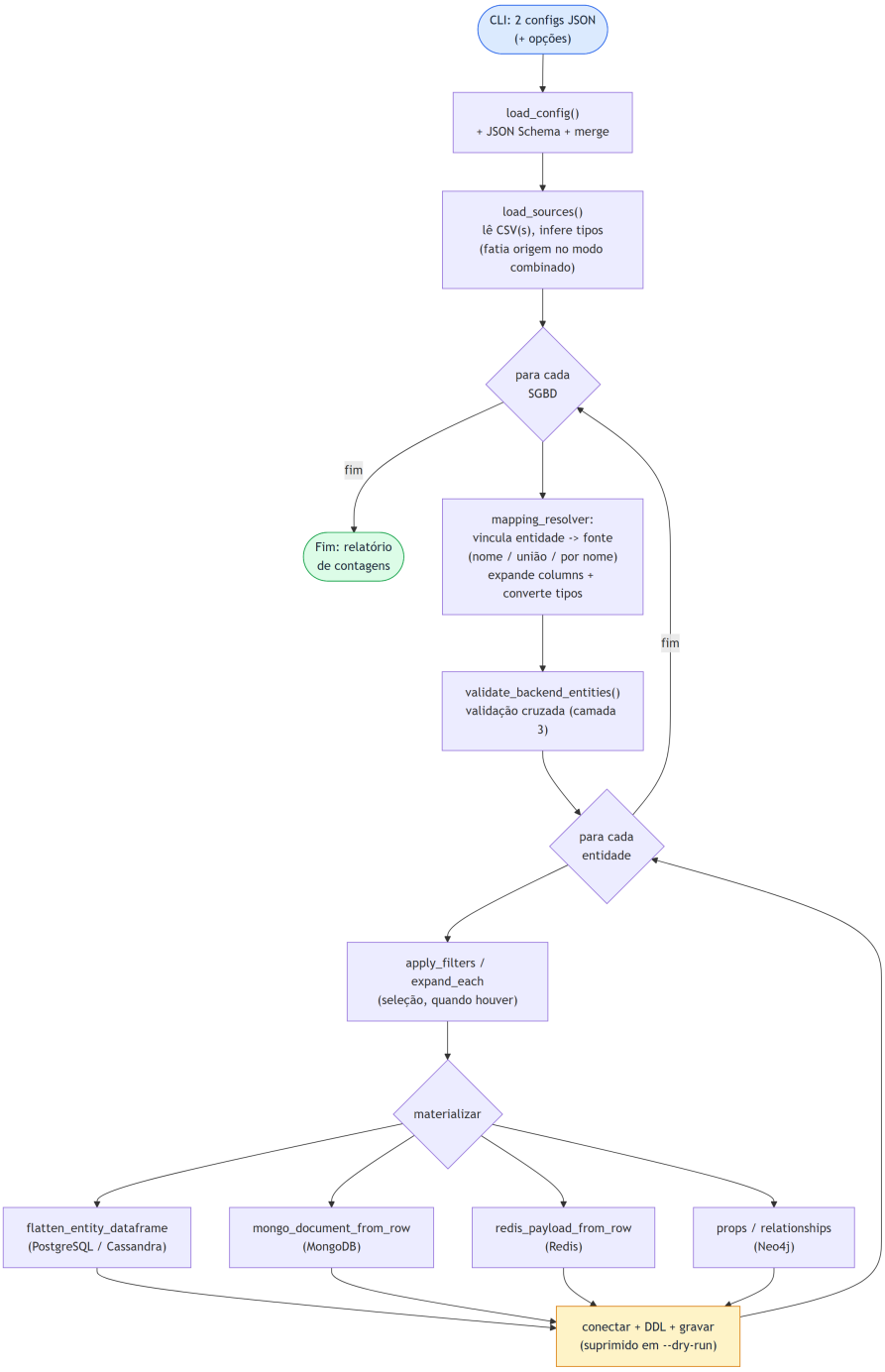


Figura 11 – Algoritmo de execução da importação PolyglotImportCSV.
Fonte: Elaborado pelo autor (2026).

A execução parte da **entrada** fornecida pelo usuário na linha de comando: os caminhos dos dois arquivos JSON de configuração (importação e conexão), eventuais sobrescritas de fonte (`--source`) e as opções de execução `--dry-run`, `--create-schema`/`--no-create-schema` e `--only`, descritas na seção 4.1.

Na etapa de **carregamento da configuração**, os dois arquivos JSON são lidos e validados contra os seus respectivos JSON Schemas (camada 1 da seção 4.2.9); em seguida, verifica-se a consistência entre os arquivos — todo SGBD referenciado no arquivo de importação deve estar declarado no arquivo de conexão (camada 2) — e as configurações são mescladas em uma estrutura única de execução.

Na etapa de **carregamento das fontes**, cada CSV declarado no bloco `sources` é lido uma única vez. Todos os valores são lidos inicialmente como texto, preservando o conteúdo original do arquivo e evitando conversões prematuras de tipo (por exemplo, em datas com fuso horário). Quando a fonte é combinada, a sua primeira coluna é interpretada como a origem e cada valor distinto dá origem a uma fonte derivada; a pseudo-coluna `_source` é preenchida em ambos os modos. Em seguida, a ferramenta infere o tipo de cada coluna de cada fonte, informação que alimenta tanto a conversão dos valores para tipos nativos quanto a geração de DDL para colunas auto-mapeadas.

Validadas as entradas, inicia-se o núcleo do algoritmo, que percorre os SGBDs presentes na configuração (excluídos os não selecionados por `--only`) e, para cada um, **vincula** as suas entidades às respectivas fontes (`mapping_resolver`): resolve o `source` de cada entidade — nome único, união de fontes ou vínculo pelo nome —, expande o mapeamento de colunas (manual, automático ou híbrido) e converte os valores da fonte para tipos nativos. Realiza-se então a validação cruzada da camada 3 (seção 4.2.9). Qualquer falha, em qualquer camada, encerra a execução com uma mensagem de erro antes que qualquer conexão com os SGBDs seja aberta.

Para cada entidade vinculada, a ferramenta **aplica os filtros** declarados na configuração, quando houver — operação que corresponde à *seleção* da álgebra relacional: apenas as linhas que satisfazem os predicados participam da entidade. Havendo filtro com o operador `each`, a entidade é adicionalmente **particionada** pelos valores distintos da coluna indicada — um particionamento horizontal em que cada valor dá origem a um destino próprio, identificado por sufixo.

Em seguida, na **materialização**, cada linha selecionada é convertida na estrutura do modelo de dados de destino, segundo os conceitos revisados no capítulo 2. Para os SGBDs de organização tabular (o relacional PostgreSQL e o colunar Cassandra), a conversão combina três operações também descritas pela álgebra relacional: *projeção* (somente as colunas mapeadas em `columns` são mantidas), *renomeação* (aplicação de `schema_column`) e *eliminação de duplicatas* com base nas colunas de

chave (`is_key`), de modo que cada tabela receba apenas tuplas únicas. Para o modelo de documentos (MongoDB), os valores da linha são agregados em um documento cuja hierarquia reproduz o aninhamento declarado em `columns`, materializando a representação desnormalizada característica desse modelo. Para o modelo chave-valor (Redis), cada linha dá origem a um par em que a chave é o valor da coluna marcada com `is_key` e o valor é a representação textual (JSON) dos demais campos mapeados. Para o modelo de grafo (Neo4j), cada linha dá origem a um nó com as propriedades mapeadas; na sequência, as arestas declaradas em `relationships` conectam os nós criados, com semântica de mesclagem — a aresta só é criada se ainda não existir, evitando duplicação quando a mesma relação aparece em várias linhas.

Por fim, para cada entidade materializada, a ferramenta **conecta-se** ao SGBD (etapa suprimida quando `--dry-run` está ativo), **cria as estruturas de armazenamento** quando `--create-schema` está ativo (tabelas no PostgreSQL; *keyspace* e tabelas no Cassandra) e **grava** os registros por meio da operação de escrita própria de cada tecnologia. A **saída** da execução é um relatório com as contagens de registros gravados por entidade e por SGBD, que permite conferir o resultado da carga.

4.3.1 O Cenário de Exemplo e os Arquivos de Dados

Para concretizar o algoritmo, esta subseção e a seguinte percorrem o cenário de e-commerce que acompanha o repositório da ferramenta — o mesmo cenário cujos fragmentos de configuração ilustraram a seção 4.2 e cuja execução completa é demonstrada na seção 4.3.3.

Os dados do cenário registram os eventos de uma loja virtual, agrupados por tipo: **reposição de estoque** (`stock`), **compra** (`purchase`), **seleção de produto** (`select_product`) e **adição ao carrinho** (`add_to_cart`). No modo multifonte — adotado como referência neste exemplo — cada tipo tem o seu próprio CSV (`ecommerce_stock.csv`, `ecommerce_purchase.csv`, `ecommerce_select_product.csv` e `ecommerce_add_to_cart.csv`), cada um com oito linhas e as colunas pertinentes ao seu assunto. Como alternativa equivalente, o modo combinado reúne os mesmos eventos em um único arquivo largo, `ecommerce_join.csv`, cuja primeira coluna (`action`) rotula a origem de cada linha; o Apêndice B reproduz esse arquivo em forma tabular, organizado por ação.

A Tabela 3 apresenta as oito linhas da fonte `stock` (`ecommerce_stock.csv`), que servem de fio condutor do exemplo. Para caber na página, a tabela resume as colunas de identificação, cujos valores seguem padrões regulares no arquivo: o produto `Product`N tem `product_id` N e pertence à categoria `Categoria`N (`category_id` N); o usuário `user`N é identificado por um `user_id` alfanumérico (por exemplo, `7fd7eee7-...` para `user1`), tem e-mail `user`N`@example.com` e um endereço próprio, detalhado no

Apêndice B; a descrição e a imagem do produto seguem padrões análogos. Todos os eventos de estoque ocorrem em 02/11/2023, e a coluna `last_restock_date` repete o instante do próprio evento.

Tabela 3 – Linhas da fonte `stock` (`ecommerce_stock.csv`, colunas de identificação resumidas).

Hora	Usuário	Produto	Variante	Marca	Categoria	Qtd.	Preço
03:30	user1	Product2	blue	Microsoft	Categoria2	2	50.5
06:45	user2	Product3	green	Google	Categoria3	1	52.07
09:15	user2	Product9	black	Pepsi	Categoria9	67	32.78
12:00	user8	Product8	white	Google	Categoria8	60	46.79
14:30	user9	Product4	yellow	Apple	Categoria4	82	55.94
17:15	user9	Product5	red	Samsung	Categoria5	57	34.21
20:00	user9	Product6	orange	Coca-Cola	Categoria6	64	51.98
22:30	user9	Product7	grey	Audi	Categoria7	1	100000

Fonte: Elaborado pelo autor (2026), a partir de `data/ecommerce/ecommerce_stock.csv`.

4.3.2 A Importação em Cada SGBD

Apresentados os arquivos de dados, esta subseção acompanha o percurso dessas linhas pelo núcleo do algoritmo, SGBD a SGBD. Em cada caso estão em jogo as entradas da ferramenta: as linhas da fonte pertinente, o trecho correspondente do arquivo de importação e o arquivo de conexão — trechos reproduzidos nas listagens da seção 4.2 e, na íntegra, no Apêndice C. O que varia de um SGBD para outro é a estrutura de dados produzida como saída, apresentada a seguir.

PostgreSQL. O arquivo de importação declara quatro entidades (Listagem 4.3). A entidade `inventory` lê da fonte `stock` e projeta quatro colunas, com o `product_id` como chave: o resultado é a relação da Tabela 4, criada e gravada no banco `ecommerce` declarado no arquivo de conexão (Listagem 4.4). As entidades `categories` e `products` também leem de `stock`, e a eliminação de duplicatas pela chave reduz o resultado aos oito produtos e oito categorias distintos; a entidade `orders` lê da fonte `purchase` (as oito compras). As chaves estrangeiras declaradas em `relationships` ligam produtos a categorias e pedidos a produtos.

Tabela 4 – Relação `inventory` materializada no PostgreSQL a partir da fonte `stock`.

product_id	quantity_available	last_restock_date	price
2	2	02/11/2023 03:30	50.5
3	1	02/11/2023 06:45	52.07
9	67	02/11/2023 09:15	32.78
8	60	02/11/2023 12:00	46.79
4	82	02/11/2023 14:30	55.94
5	57	02/11/2023 17:15	34.21
6	64	02/11/2023 20:00	51.98
7	1	02/11/2023 22:30	100000

Fonte: Elaborado pelo autor (2026).

MongoDB. A entidade `product_catalog` (Listagem 4.5) também lê da fonte `stock`, mas materializa cada linha como um documento cuja hierarquia reproduz o aninhamento declarado em `columns`. A Listagem 4.13 mostra o documento gerado a partir da primeira linha de estoque. Os valores preservam a forma textual lida do CSV, coerente com a política de carregamento descrita no início desta seção; os subdocumentos `category` e `stock` agrupam, sem repetição de linhas, o que no PostgreSQL ficou distribuído em três tabelas ligadas por chaves estrangeiras — a representação desnormalizada característica do modelo de documentos.

```
{
  "product_id": "2",
  "product_name": "Product2",
  "product_variant": "blue",
  "product_brand": "Microsoft",
  "product_description": "Description1",
  "product_image": "https://example.com/image1.jpg",
  "price": "50.5",
  "category": {
    "category_id": "2",
    "category_name": "Categoria2"
  },
  "stock": {
    "quantity_available": "2",
    "last_restock_date": "2023-11-02 03:30:00Z"
  }
}
```

Listagem 4.13 – Documento gerado na coleção `product_catalog` para a primeira linha de estoque.

Cassandra. A entidade `user_activity_log` (Listagem 4.7) une as quatro fontes: os 32 eventos alimentam a tabela, com `timestamp` renomeado para `event_time` e

a pseudo-coluna `_source` mapeada para `event_type`. A chave primária composta — partição por `user_id`, agrupamento por `timestamp` — faz com que todos os eventos de um mesmo usuário fiquem na mesma partição, ordenados no tempo. A Tabela 5 mostra a partição do usuário `user1`: os seus quatro eventos, um de cada fonte, com as colunas de referência preenchidas conforme a origem — cada fonte preenche apenas o subconjunto de colunas pertinente ao seu evento, agora a serviço de uma consulta típica do modelo colunar ("o histórico de atividades de um usuário").

Tabela 5 – Partição do usuário `user1` na tabela `user_activity_log` (Cassandra).

event_time	event_type	product_id	order_number	selected_product_id	shopping_cart_id
02/11 03:30	stock	2	—	—	—
15/11 15:09	select_product	—	—	11	—
15/11 15:17	add_to_cart	—	—	—	user:7fd7...:cart
15/11 15:46	purchase	2	ORD1	—	—

Fonte: Elaborado pelo autor (2026).

Redis. As duas entidades desse SGBD (Listagem 4.9) leem de fontes distintas: `shopping_cart` da fonte `add_to_cart` e `user_session` da fonte `select_product`. Cada linha torna-se um par chave-valor: a chave é o valor da coluna marcada com `is_key` e o valor é a representação textual (JSON) dos demais campos mapeados. A Listagem 4.14 mostra os dois pares gerados a partir dos eventos de `user1`; ao todo, são oito carrinhos e oito sessões. Como `shopping_cart` usa mapeamento automático, o seu valor reúne todas as demais colunas da fonte `add_to_cart`; já `user_session` mapeia apenas os campos declarados. Note-se que o padrão `user:...:cart` da chave do carrinho vem do próprio CSV (coluna `shopping_cart_id`), e não de alguma convenção da ferramenta.

```
chave: user:7fd7eee7-4675-4f56-ad81-108da6cecebc:cart
valor: {"timestamp": "2023-11-15 15:17:30Z", "user_id": "7fd7...",
        "user_name": "user1", "user_email": "user1@example.com",
        "cart_product_id": "3", "cart_quantity": "1"}
chave: 7fd7eee7-4675-4f56-ad81-108da6cecebc
valor: {"user_name": "user1", "user_email": "user1@example.com",
        "last_seen": "2023-11-15 15:09:30Z"}
```

Listagem 4.14 – Pares chave-valor gerados no Redis a partir dos eventos de `user1`.

Neo4j. As entidades e a aresta da Listagem 4.11 produzem um grafo de propriedades: os oito produtos da fonte `stock` dão origem aos nós `Product`; os oito com-

pradores da fonte `purchase` dão origem aos nós `User` e às oito arestas `PURCHASED`, identificadas pelo número do pedido e decoradas com quantidade, preço e nota da avaliação. A semântica de mesclagem garante que nós e arestas não se dupliquem quando a mesma chave reaparece. A Listagem 4.15 representa, na notação usual de grafos de propriedades, o fragmento do grafo derivado da primeira compra (`ORD1`).

```
(:User {user_id: "7fd7eee7-...", user_name: "user1",
        user_email: "user1@example.com"})
    -[:PURCHASED {order_number: "ORD1", quantity: "1",
                  price: "50.5", rating: "2"}]->
(:Product {product_id: "2", product_name: "Product2",
           product_brand: "Microsoft"})
```

Listagem 4.15 – Fragmento do grafo materializado no Neo4j para o pedido `ORD1`.

A Tabela 6 consolida o percurso, tomando por referência a primeira linha da fonte `stock` (Tabela 3, o produto `Product2`): cada célula da coluna “Saída materializada” decorre diretamente das entidades, do vínculo com a fonte e das `columns` declaradas no arquivo de importação.

Tabela 6 – Materialização de uma linha da fonte `stock` nos diferentes SGBDs.

SGBD	**Entidade**	**Fonte**	**Saída materializada**
PostgreSQL	`inventory` (a linha também alimenta `products` e `categories`)	`stock`	Linha da Tabela 4: `product_id`, `quantity_available`, `last_restock_date`, `price`.
MongoDB	`product_catalog`	`stock`	Documento da Listagem 4.13, com subdocumentos `category` e `stock` aninhados.
Cassandra	`user_activity_log`	`stock` ∪ demais	Registro com `timestamp` e `_source` renomeados para `event_time` e `event_type`; chave primária composta `(user_id, timestamp)`.
Redis	—	—	As entidades Redis leem de outras fontes; esta linha não alimenta o Redis.
Neo4j	nó `Product`	`stock`	Nó `:Product` da Listagem 4.15, identificado por `product_id`.

Fonte: Elaborado pelo autor (2026).

Assim, **fontes heterogêneas** alimentam destinos igualmente heterogêneos: tabelas relacionais, documentos aninhados, *log* colunar com renomeação de colunas e união de fontes, pares chave-valor e um grafo de propriedades — materializando a persistência poliglota descrita no capítulo 2. As contagens observadas neste percurso (oito registros por entidade de estoque e 32 no registro de atividades) reaparecem na verificação prévia da execução, apresentada a seguir.

4.3.3 Exemplo de Uso e Evidência de Execução (Cenário E-commerce)

Esta seção demonstra o uso da ferramenta em uma execução completa do cenário de e-commerce que acompanha o código-fonte. Todos os arquivos de entrada encontram-se no diretório `data/ecommerce/` do repositório da ferramenta: os

quatro CSVs de fonte (`ecommerce_stock.csv`, `ecommerce_purchase.csv`, `ecommerce_select_product.csv` e `ecommerce_add_to_cart.csv`), com oito linhas cada; o arquivo de importação `import_config.json`, cujos fragmentos por SGBD foram apresentados nas Listagens 4.3 a 4.11; e o arquivo de conexão `sgbd_config.json`, cujos fragmentos acompanham as mesmas subseções — ambos reproduzidos na íntegra no Apêndice C. Além deles, o arquivo `docker-compose.yml`, na raiz do repositório, define os contêineres dos cinco SGBDs usados no exemplo. A execução usa o comando apresentado na seção 4.1, instanciado com esses arquivos:

```
python -m polyglotimportcsv \
  --config data/ecommerce/import_config.json \
  --sgbd-config data/ecommerce/sgbd_config.json \
  --dry-run
```

Com a *flag* `--dry-run`, a ferramenta executa todas as validações e a materialização, mas não abre conexão com nenhum SGBD — o *driver* Apache Cassandra, por exemplo, sequer é importado, o que permite o *dry-run* em ambientes onde a sua extensão C não está disponível. O resultado são as contagens de registros por destino mostradas na Tabela 7, úteis para revisar o mapeamento antes da carga real.

Tabela 7 – Contagens reportadas pelo modo `--dry-run` no cenário e-commerce.

SGBD	**Destino**	**Registros (após projeção/deduplicação)**
PostgreSQL	`categories`	8
PostgreSQL	`products`	8
PostgreSQL	`inventory`	8
PostgreSQL	`orders`	8
MongoDB	`product_catalog`	8 documentos
Cassandra	`user_activity_log`	32 linhas (união das 4 fontes)
Redis	`shopping_cart`	8
Redis	`user_session`	8
Neo4j	nó `User`	8
Neo4j	nó `Product`	8
Neo4j	aresta `PURCHASED`	8 (uma por linha da fonte `purchase`)

Fonte: Elaborado pelo autor (2026).

Para a importação real, basta subir os contêineres com `docker compose up -d` e repetir o mesmo comando sem a *flag* `--dry-run`: os mesmos volumes da Tabela 7 são então persistidos nos cinco SGBDs configurados, materializando o cenário poliglota da seção 2.1. Alternativamente, o script `run_example.sh`, na raiz do repositório, automatiza

o fluxo completo: lê o `sgbd_config.json`, sobe apenas os contêineres dos SGBDs ali declarados e, em seguida, executa o *dry-run* e a importação real com `--create-schema`.

Complementarmente à evidência de execução, testes automatizados (`pytest`) cobrem a validação por schema, a separação e mesclagem dos dois arquivos de configuração, o carregamento das fontes (multifonte e combinada), o mapeamento de colunas (`csv_column`, `schema_column`, aninhamento, mapeamento automático), a união de fontes, a inferência e conversão de tipos, os filtros e *smoke tests* de validação com *dry-run*. Um teste dedicado verifica ainda a **equivalência entre os modos** multifonte e combinado, assegurando que ambos produzem a mesma carga.

4.4 OBSERVABILIDADE E MÉTRICAS

Importar dados para cinco SGBDs em uma só execução exige que o usuário consiga **acompanhar** o que ocorre e **diagnosticar** falhas sem recorrer ao código-fonte. Para isso, a ferramenta incorpora uma camada de observabilidade, construída sobre a biblioteca `rich`, e um subsistema de métricas de desempenho. Além de servir ao uso cotidiano, esse subsistema é o **instrumento de medição** empregado na avaliação de desempenho do Capítulo 5.

Relato da execução. Cada execução emite, no terminal, um relato estruturado: um cabeçalho com o modo (importação ou *dry-run*), as fontes carregadas com as suas contagens de linhas e colunas, uma seção por SGBD com o progresso de cada entidade e um resumo final. O nível de detalhe das mensagens é ajustável por `--log-level` (`DEBUG`, `INFO`, `WARNING` ou `ERROR`; padrão `INFO`). Independentemente do nível escolhido para o terminal, a ferramenta grava um **arquivo de *log* de sessão** sempre em nível `DEBUG`, em `logs/polyglotimportcsv_<carimbo de tempo>.log`, preservando o rastro completo de cada execução para diagnóstico posterior; esse comportamento pode ser desativado ou redirecionado por variáveis de ambiente. Para inspeção dos dados, a ferramenta exibe uma amostra de cada entidade materializada até um limite de 50 linhas — exibição que `--show-data` força e `--no-data` suprime.

Métricas de desempenho. Ao longo da execução, a ferramenta coleta métricas por **fase** de processamento de cada entidade, distinguindo quatro fases: **leitura** (`read`, o carregamento das fontes), **seleção** (`filter`, a aplicação de filtros), **mapeamento** (`map`, o vínculo com a fonte e a conversão de tipos) e **escrita** (`write`, a persistência no SGBD). Para cada fase registram-se o número de linhas processadas e o tempo decorrido, do que se deriva a vazão em linhas por segundo. Ao final, um quadro-resumo apresenta essas métricas no terminal. Quando a *flag* `--benchmark` está ativa, a ferramenta grava as métricas em disco — um arquivo JSON com carimbo de tempo e uma linha acrescentada a um CSV histórico, no diretório `benchmarks/` —, acompanhadas

de metadados do ambiente de execução (versão do Python, sistema operacional e tamanho de cada fonte). Além do tempo por fase, o instrumento registra o **pico de memória** consumido durante a execução, coletado com a biblioteca `tracemalloc`; essa medida é o que permite comparar, no Capítulo 5, o custo de memória dos dois modelos de execução apresentados na seção seguinte. Essa gravação estruturada é o que viabiliza a coleta sistemática de resultados reproduzíveis, explorada no Capítulo 5.

4.5 MODELOS DE EXECUÇÃO E ESTRATÉGIAS DE ESCRITA

A forma como a ferramenta **converte** os valores lidos e os **grava** no SGBD não é única: a importação é parametrizada por dois eixos independentes, expostos na CLI e com um valor-padrão eficiente. O primeiro, a **estratégia de escrita** (`--strategy`), decide se a conversão de tipos e a gravação ocorrem linha a linha ou em lote. O segundo, o **modelo de execução** (`--execution`), decide se o arquivo é carregado por inteiro na memória antes de ser gravado ou se é processado em fluxo, com consumo de memória limitado. Ambos os eixos são os fatores controlados na avaliação de desempenho do Capítulo 5; esta seção descreve o que cada um significa em termos de projeto e implementação.

4.5.1 Estratégias de Escrita

A opção `--strategy` admite dois valores. Na estratégia **ingênua** (`naive`), a ferramenta percorre os registros **um a um**: cada célula é convertida ao tipo nativo por uma chamada individual e cada linha é gravada no SGBD por um comando próprio. Esse caminho é simples e serve de **referência de comparação**, mas paga, por linha, o custo de uma conversão isolada e de uma ida e volta à rede (e, quando aplicável, de uma confirmação de transação). Na estratégia **otimizada** (`optimized`, o padrão), esses dois custos são amortizados. A conversão de tipos passa a ser **vetorizada** — realizada uma vez por coluna, sobre todo o conjunto de valores, em vez de célula a célula —, produzindo exatamente os mesmos valores e tipos que o caminho ingênuo. A gravação passa a ser feita em **lote**: um único comando carrega muitos registros de uma vez, reduzindo drasticamente o número de idas e vindas à rede.

Cada SGBD implementa o lote com o recurso idiomático do seu respectivo controlador (*driver*): no **PostgreSQL**, a inserção em lote de `execute_values`; no **MongoDB**, a inserção múltipla de `insert_many`; no **Cassandra**, a execução concorrente de comandos preparados (com grau de concorrência 64 dentro de cada lote); no **Redis**, o enfileiramento de comandos em um *pipeline* despachado de uma só vez; e no **Neo4j**, a cláusula `UNWIND` aplicada a uma lista de registros, seguida de `MERGE`, sobre um rótulo com restrição de unicidade. Como as duas estratégias produzem resultados idênticos,

diferindo apenas no custo, a estratégia ingênua tem valor sobretudo metodológico: é a *linha de base* contra a qual o ganho da via otimizada é medido no Capítulo 5.

4.5.2 Modelos de Execução: Materialização e *Streaming*

A opção `--execution` controla **quando** os dados ocupam a memória. No modo de **materialização** (`materialize`), cada fonte é carregada integralmente em memória — em uma estrutura tabular (*DataFrame*) — e só então percorre as quatro fases de processamento (leitura, seleção, mapeamento e escrita) descritas na seção 4.4. Esse modelo torna as fases nitidamente separáveis, razão pela qual é ele que sustenta a decomposição por fase medida na avaliação; em contrapartida, o seu **pico de memória cresce com o tamanho do arquivo**, o que o inviabiliza para entradas maiores que a memória disponível.

No modo de **fluxo** (`stream`, o padrão para importações reais), a fonte é lida em **blocos** de tamanho fixo (8192 linhas por bloco) e cada bloco é processado e gravado antes de o próximo ser lido. Assim, o pico de memória permanece próximo de **um único bloco** — somado aos poucos registros ainda acumulados à espera de completar um lote de gravação —, **constante em relação ao tamanho total do arquivo**. Isso habilita a importação de arquivos maiores que a memória da máquina, ao custo de um processamento mais entrelaçado: as quatro fases deixam de ser etapas sequenciais distintas e passam a se repetir a cada bloco.

A arquitetura que viabiliza esse modo segue o padrão **portas e adaptadores** (*ports and adapters*). Um **orquestrador** agnóstico de SGBD coordena o fluxo, mas não conhece nenhum banco em particular: ele não emite SQL nem Cypher e não remove duplicatas. Toda a especificidade de cada SGBD é isolada atrás de uma **porta** de escrita comum, o `DbmsSink`, para a qual cada banco fornece um **adaptador** (`PostgresSink`, `MongoSink`, `CassandraSink`, `RedisSink` e `Neo4jSink`). Cada adaptador reaproveita as mesmas rotinas de formatação de linha e de gravação em lote já usadas na importação por materialização, de modo que a lógica de cada SGBD não é duplicada entre os dois modelos de execução.

O processamento de cada bloco reproduz, em pequena escala, o pipeline de importação. Ao ver o primeiro bloco de uma entidade, o orquestrador realiza um **vínculo tardio**: infere os tipos das colunas a partir daquela amostra — prevalecendo sempre um tipo declarado explicitamente na configuração sobre o inferido — e reutiliza esse vínculo para os blocos seguintes. A cada bloco, aplica os filtros de seleção, converte os tipos (sempre pela via otimizada, vetorizada) e roteia as linhas para as suas partições de destino, acumulando-as em áreas de espera; quando uma partição acumula registros suficientes, um lote é descarregado ao adaptador correspondente. Duas particularidades merecem nota: a gravação de **relacionamentos** no Neo4j exige

um **segundo passe** sobre as fontes (as arestas só podem ser criadas depois que todos os nós de destino existem), realizado também em fluxo e com memória limitada; e o modo de fluxo suporta **fontes-união**, em que uma mesma entidade é alimentada por mais de um CSV.

Por fim, os dois eixos se articulam com o restante da ferramenta. A estratégia ingênua só faz sentido no modo de materialização, que reproduz a referência linha a linha; o modo de fluxo emprega sempre a via otimizada. A equivalência dos dados gravados entre os dois modelos de execução — mesmas partições, mesmas contagens e mesmas conversões — é assegurada por testes automatizados, de modo que a escolha entre materialização e fluxo é uma decisão de **custo**, não de **correção**. É justamente esse custo — tempo de importação e, sobretudo, pico de memória — que a avaliação do Capítulo 5 quantifica, varrendo sistematicamente os dois eixos aqui descritos.

5 AVALIAÇÃO DE DESEMPENHO

Este capítulo avalia empiricamente o desempenho da PolyglotImportCSV. A avaliação foi listada como objetivo específico no Capítulo 1 e apoia-se diretamente no instrumento de medição descrito na seção 4.4 — o subsistema de métricas, que cronometra cada fase da importação e registra o pico de memória por execução — e nos dois eixos de configuração descritos na seção 4.5: a estratégia de escrita (`naive` *versus* `optimized`) e o modelo de execução (`materialize` *versus* `stream`). O objetivo não é eleger o "melhor" SGBD — os cinco atendem a modelos de dados distintos — mas caracterizar o custo de importar um mesmo conjunto de dados para cinco paradigmas e quantificar o ganho de cada uma das duas estratégias de desempenho da ferramenta.

A prática de comparar sistemas de dados sob uma carga sintética controlada e reproduzível segue a tradição consolidada por Cooper et al. (2010) com o *Yahoo! Cloud Serving Benchmark* (YCSB), que estabeleceu o gerador de carga determinístico e as métricas de *throughput* como base para avaliar sistemas de dados heterogêneos. No contexto específico de persistência poliglota, Landuyt et al. (2023) e Ye et al. (2023) conduzem comparações de desempenho entre múltiplos modelos de dados sob uma carga comum — o mesmo princípio que orienta este capítulo, aqui aplicado à etapa de **importação** e não à de consulta.

5.1 METODOLOGIA

A avaliação é organizada em torno de seis **questões de pesquisa** (QP), que os resultados da seção 5.3 respondem:

- **QP1 — Custo por SGBD.** Qual é o *throughput* de escrita de cada SGBD e como o tempo total de uma importação se decompõe entre as fases de leitura, seleção, mapeamento e escrita?
- **QP2 — Modos de importação.** Importar a partir de um CSV por entidade (modo multifonte) ou de um único CSV combinado impõe sobrecustos distintos?
- **QP3 — Escalabilidade.** Como o tempo e a memória evoluem ao multiplicar o volume de dados por dez, de mil a cem mil linhas?
- **QP4 — Estratégia de escrita.** Qual é o ganho de desempenho da estratégia otimizada (vetorização e escrita em lote) sobre a ingênua (linha a linha)?

- **QP5 — Modelo de execução.** Qual é o custo, em tempo e em pico de memória, do modo de fluxo em relação ao de materialização?
- **QP6 — Memória limitada.** O pico de memória do modo de fluxo permanece constante em relação ao tamanho do arquivo, como prevê o seu projeto?

5.1.1 Geração dos Dados

A carga de trabalho é produzida por um **gerador determinístico** que sintetiza um domínio de comércio eletrônico a partir de um único parâmetro de escala N (o número de produtos). As demais fontes crescem proporcionalmente a N: o estoque tem N linhas, as compras $3N$, as seleções $2N$ e os carrinhos $2N$. Todos os valores derivam de sementes textuais fixas, de modo que a mesma escala N produz **sempre os mesmos arquivos**, byte a byte, preservando a integridade referencial entre as fontes (todo produto referenciado em uma compra existe no estoque). Essa reprodutibilidade é o que permite comparar campanhas medidas em momentos diferentes: como a carga é idêntica, apenas o tempo e a memória variam entre execuções. O uso de um gerador sintético parametrizável, em vez de um conjunto de dados fixo, segue a mesma orientação metodológica do YCSB (COOPER et al., 2010).

5.1.2 Protocolo de Medição

Cada célula experimental é definida por uma combinação de **tamanho** ($N \in \{1000, 10\,000, 100\,000\}$), **modo** (multifonte ou combinado), **estratégia** (`naive` ou `optimized`), **modelo de execução** (`materialize` ou `stream`) e é repetida **três vezes**. Antes de cada repetição, os cinco SGBDs são **limpos** (*cold-load*): as estruturas de destino são truncadas, de modo que nenhuma repetição se beneficia de dados ou de caches deixados pela anterior. O valor reportado para cada célula é a **mediana** das três repetições, que é robusta a uma eventual repetição anômala.

O instrumento de medição impõe uma restrição metodológica importante. O tempo por fase é cronometrado com um relógio de alta resolução; já o pico de memória é obtido instrumentando as alocações do interpretador Python. Essa instrumentação de memória, porém, **infla o tempo** das fases de leitura e mapeamento (cerca de $8{,}6\times$ a leitura e $6{,}5\times$ o mapeamento), enquanto quase não afeta a escrita nos bancos — distorcendo, portanto, as fases umas em relação às outras. Por isso, tempo e memória **não podem ser lidos da mesma execução**. Adotou-se um **protocolo pareado**: duas campanhas idênticas em carga e configuração, uma **com** a instrumentação de memória ativada (fonte dos picos de memória) e outra **sem** ela (fonte dos tempos). As duas campanhas produziram contagens de linhas idênticas em todas as células, confirmando que executam a mesma carga. Ao longo do capítulo, **cada tabela e figura declara**

na legenda de qual das duas campanhas os seus números provêm, e valores de tempo e de memória nunca são combinados em uma mesma linha.

Quanto à precisão, duas campanhas de tempo da mesma célula, sobre o mesmo código, chegaram a diferir entre 15% e 38%: as medidas de tempo carregam, portanto, uma incerteza da ordem de 20%, e diferenças menores que isso não sustentam conclusões. O pico de memória, ao contrário, reproduziu-se com diferença inferior a $0{,}1\%$ entre campanhas independentes, sendo a métrica mais estável do estudo.

5.2 CONFIGURAÇÃO EXPERIMENTAL

Todos os experimentos foram executados em uma **única máquina** com sistema operacional Windows 11 (*build* 10.0.26200) e Python 3.14.5. A semente global de geração foi fixada em 42 e cada célula foi repetida três vezes. Os cinco SGBDs foram providos por contêineres Docker, nas versões indicadas na Tabela 8. Os contêineres partilham os recursos do mesmo hospedeiro (8 GB de memória física); em particular, o Cassandra opera com um *heap* de 1 GB dentro de uma máquina virtual Docker de cerca de 3,8 GB — uma restrição de ambiente que se mostrará relevante na análise da fase de escrita.

Tabela 8 – Imagens Docker dos SGBDs utilizadas na avaliação.

SGBD	**Imagem Docker**
PostgreSQL	`postgres:16-alpine`
Redis	`redis:7-alpine`
MongoDB	`mongo:7`
Cassandra	`cassandra:4.1`
Neo4j	`neo4j:5-community`

Fonte: Elaborado pelo autor (2026), a partir de `docker-compose.yml`.

A matriz completa combina os três tamanhos, os dois modos, as duas estratégias e os dois modelos de execução. Nem toda combinação é medida em todos os pontos: a estratégia `naive` serve de linha de base e foi medida apenas no ponto $N = 10\,000$, no modo de materialização, pois o modo de fluxo emprega sempre a via otimizada por projeto (seção 4.5.2). Cada resultado a seguir indica o seu escopo.

5.3 RESULTADOS

5.3.1 Estratégia de Escrita: Otimizada *versus* Ingênua (QP4)

A Tabela 9 contrasta as duas estratégias de escrita no ponto $N = 10\,000$, no modo de materialização. A via otimizada importa o mesmo conjunto de dados — as mesmas 27.494 linhas gravadas — entre $8{,}5$ e $9{,}3$ vezes mais rápido que a via ingênua. O ganho concentra-se, como esperado, nas duas fases que a otimização ataca: o mapeamento (conversão de tipos), acelerado cerca de $42\times$ pela vetorização, e a escrita, acelerada $7{,}7\times$ pela gravação em lote. A leitura e a seleção, que a estratégia não altera, permanecem estáveis.

Tabela 9 – Tempo total e das fases dominantes por estratégia de escrita, em segundos ($N = 10\,000$, materialização). Campanha sem instrumentação de memória.

Modo	Estratégia	Mapeamento	Escrita	Total	Ganho
multifonte	ingênua	38,54	95,36	135,74	—
	otimizada	0,91	12,44	14,54	$9{,}3\times$
combinado	ingênua	—	—	138,44	—
	otimizada	—	—	16,30	$8{,}5\times$

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings-naive/`.

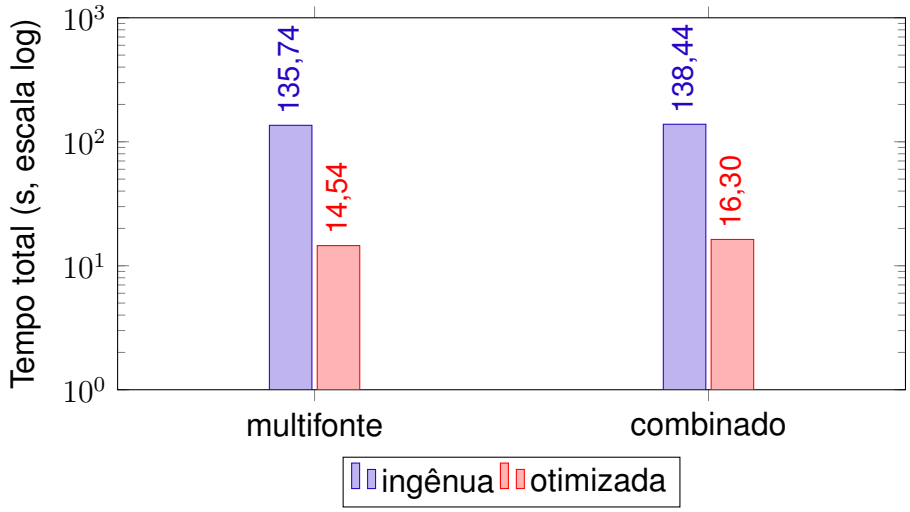


Figura 12 – Tempo total de importação por estratégia de escrita ($N = 10\,000$, materialização; eixo vertical em escala logarítmica). Campanha sem instrumentação de memória.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings-naive/`.

Dado esse ganho, e por a estratégia otimizada ser o padrão da ferramenta, todos os demais resultados deste capítulo empregam `optimized`. A estratégia ingênua

cumpre aqui o seu papel exclusivamente metodológico: a linha de base que dá sentido ao número "$9{,}3\times$".

5.3.2 Decomposição por Fase e Escrita por SGBD (QP1)

Fixada a estratégia otimizada, a Tabela 10 decompõe uma importação de $N = 100\,000$ linhas (multifonte, materialização) nas suas quatro fases. A escrita nos bancos domina o custo, respondendo por cerca de 91% do tempo total; a leitura e o mapeamento, juntos, respondem por menos de 10%, e a seleção é desprezível.

Tabela 10 – Tempo por fase de uma importação otimizada ($N = 100\,000$, multifonte, materialização), em segundos. Campanha sem instrumentação de memória.

Fase	**Tempo (s)**	**Participação**
Leitura	7,69	5,1%
Seleção	0,00	0,0%
Mapeamento	5,90	3,9%
Escrita	136,34	91,0%
Total	149,93	100%

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

Como a escrita concentra o custo, convém abri-la por SGBD. A Tabela 11 e a Figura 13 apresentam, para a mesma importação, o número de linhas gravadas, o tempo de escrita e o *throughput* resultante em cada um dos cinco bancos. Os valores variam por mais de uma ordem de grandeza: o MongoDB grava a cerca de 14.431 linhas/s, enquanto o Cassandra fica em 912 linhas/s.

Tabela 11 – Escrita por SGBD em uma importação otimizada ($N = 100\,000$, multifonte, materialização). Campanha sem instrumentação de memória.

SGBD	**Linhas**	**Tempo (s)**	**Linhas/s**
Cassandra	100.000	109,61	912
Neo4j	51.602	13,23	3.901
PostgreSQL	62.167	8,13	7.644
Redis	49.558	4,56	10.865
MongoDB	11.609	0,80	14.431

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

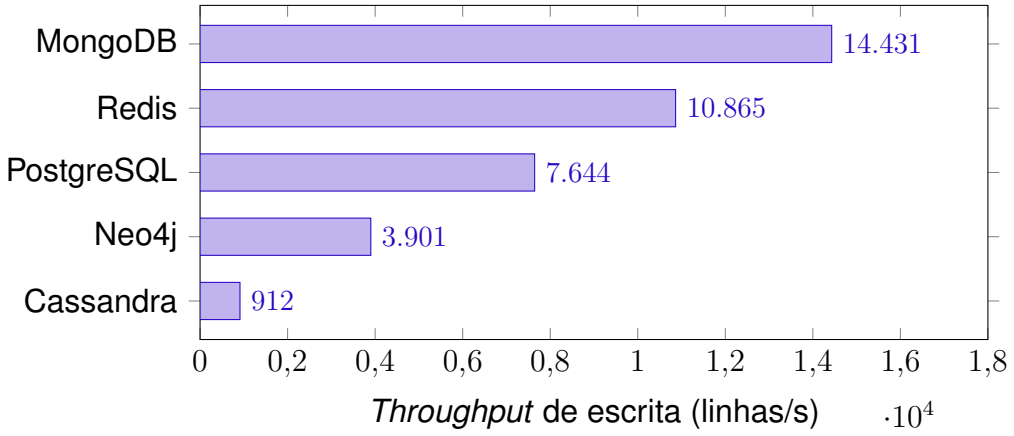


Figura 13 – *Throughput* de escrita por SGBD ($N = 100\,000$, multifonte, materialização, estratégia otimizada). Campanha sem instrumentação de memória.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

O Cassandra sozinho responde por $109{,}6$ s dos $136{,}3$ s da fase de escrita — cerca de 80%. Qualquer afirmação sobre "o tempo de escrita" é, portanto, em larga medida uma afirmação sobre o Cassandra neste ambiente, o que reforça a necessidade de abrir a fase por SGBD em vez de tratá-la como um bloco único.

5.3.3 Modos de Importação: Multifonte *versus* Combinado (QP2)

Os dois modos de importação — um CSV por entidade ou um único CSV combinado — gravam os mesmos dados e, como mostra a Tabela 12, têm desempenho muito próximo. No ponto de $100\,000$ linhas, o modo combinado é cerca de 8% mais lento em tempo e usa cerca de 6% mais memória — diferenças da ordem da própria incerteza de medição do tempo ($\approx 20\%$). Na prática, os dois modos são intercambiáveis em custo, e a escolha entre eles é de conveniência de organização dos dados, não de desempenho.

Tabela 12 – Multifonte *versus* combinado ($N = 100\,000$, estratégia otimizada). Tempos e picos de memória provêm de campanhas distintas, conforme indicado.

	Materialização		**Fluxo**	
Modo	**Tempo (s)**	**Pico (MB)**	**Tempo (s)**	**Pico (MB)**
multifonte	149,93	950,7	165,03	75,9
combinado	161,94	$1.009{,}5$	181,42	79,8

Fonte: Elaborado pelo autor (2026). Tempo de `benchmarks/timings/`; memória de `benchmarks/`.

5.3.4 Modelos de Execução: Memória e Tempo (QP5 e QP6)

O resultado central da avaliação está no contraste entre materialização e fluxo. A Tabela 13 e a Figura 14 mostram o pico de memória dos dois modelos em função do tamanho do arquivo, no modo multifonte. A materialização cresce **linearmente** com o volume: de $10{,}5$ MB em mil linhas para $950{,}7$ MB em cem mil — praticamente $90\times$ mais memória para $100\times$ mais dados. O fluxo, ao contrário, **achata**: de $10{,}7$ MB para $75{,}9$ MB no mesmo intervalo. Em cem mil linhas, o fluxo usa $12{,}5\times$ **menos memória** que a materialização, e a vantagem cresce com o tamanho do arquivo. Esse achatamento é a evidência empírica que responde à QP6: o pico do modo de fluxo é governado pelo tamanho do bloco, não pelo do arquivo, exatamente como prevê o seu projeto (seção 4.5.2).

Tabela 13 – Pico de memória por modelo de execução (N crescente, multifonte, estratégia otimizada), em MB. Campanha com instrumentação de memória.

Linhas	Materialização	Fluxo	Razão
1.000	10,5	10,7	$0{,}98\times$
10.000	95,3	47,8	$2{,}0\times$
100.000	950,7	75,9	$\mathbf{12{,}5}\times$

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/` (raiz).

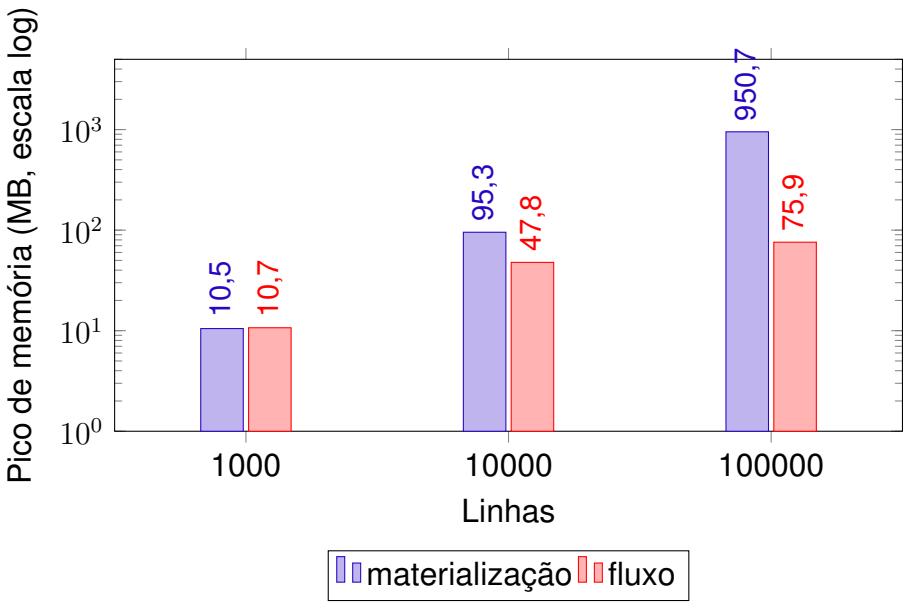


Figura 14 – Pico de memória por modelo de execução em função do volume (multifonte, escala logarítmica): a materialização cresce com o arquivo; o fluxo achata.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/` (raiz).

Essa economia de memória tem um custo em tempo, mas modesto e **decres-**

cente. A Tabela 14 e a Figura 15 mostram que o sobrecusto de tempo do fluxo cai de $+61\%$ em mil linhas para apenas $+10\%$ em cem mil. A razão é que, à medida que o volume cresce, a escrita nos bancos — comum aos dois modelos — passa a dominar o tempo total, diluindo o custo fixo do processamento entrelaçado do fluxo. Ou seja: **quanto maior o arquivo, mais barata fica a troca** — a vantagem de memória do fluxo cresce justamente onde o seu custo de tempo encolhe.

Tabela 14 – Tempo total por modelo de execução (N crescente, multifonte, estratégia otimizada), em segundos. Campanha sem instrumentação de memória.

Linhas	Materialização	Fluxo	Sobrecusto
1.000	2,87	4,63	$+61\%$
10.000	16,20	20,94	$+29\%$
100.000	149,93	165,03	$+10\%$

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

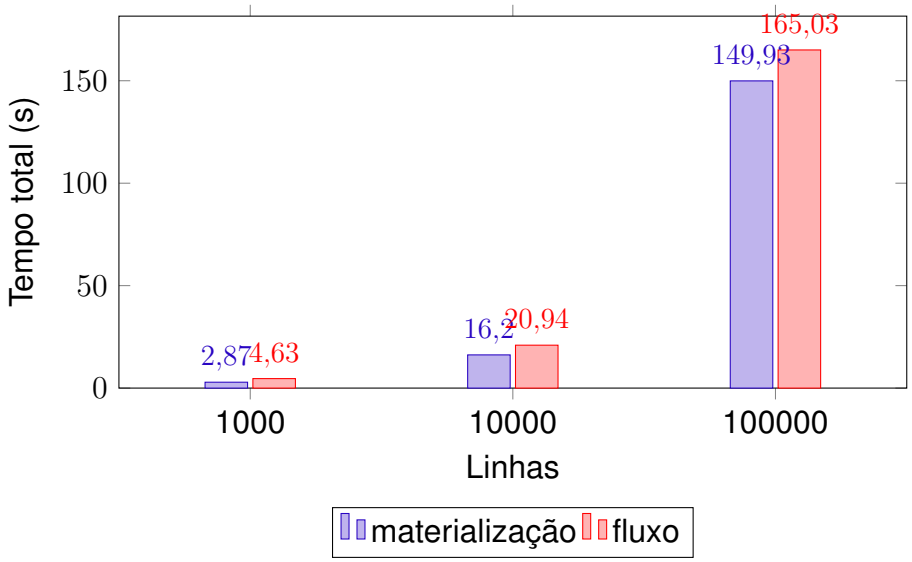


Figura 15 – Tempo total por modelo de execução em função do volume (multifonte): o sobrecusto do fluxo encolhe de $+61\%$ para $+10\%$ à medida que a escrita passa a dominar.

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/timings/`.

5.3.5 Uma Observação sobre o Pico de Memória da Estratégia Otimizada

Um ponto exige cuidado na leitura das medidas de memória. No mesmo ponto de $10\,000$ linhas e modo de materialização, a estratégia **ingênua** exibe um pico **menor** que a otimizada — $14{,}4$ MB contra $95{,}4$ MB no modo multifonte (Tabela 15). O número não é um erro, e reproduz-se entre campanhas. A explicação está na natureza da otimização: para gravar em lote, o caminho otimizado **acumula registros em memória**

antes de despachá-los. No Neo4j, em particular, a gravação de relacionamentos por `UNWIND` monta a lista completa de arestas de uma entidade em memória antes de submetê-la, ao passo que a via ingênua percorre as arestas uma a uma, sem acumular. A escrita em lote, portanto, **troca memória por vazão**: ela é $7{,}7\times$ mais rápida (Tabela 9), mas ao preço de um pico de memória maior.

Tabela 15 – Pico de memória por estratégia de escrita ($N = 10\,000$, materialização), em MB. Campanha com instrumentação de memória.

Modo	**Ingênua**	**Otimizada**
multifonte	14,4	95,4
combinado	21,3	$101{,}6$

Fonte: Elaborado pelo autor (2026), a partir de `benchmarks/naive/`.

Vale notar que esse acúmulo é justamente o que o **modo de fluxo** limita: ao processar o arquivo em blocos, ele mantém em memória apenas um bloco e os lotes ainda não descarregados — e é por isso que o fluxo, mesmo empregando sempre a via otimizada, atinge picos muito menores que a materialização otimizada (Tabela 13). Os dois eixos, portanto, atuam sobre a memória de formas complementares.

5.4 ANÁLISE E DISCUSSÃO

Os resultados respondem às seis questões de pesquisa. A escrita nos bancos é a fase dominante de uma importação (QP1): mais de 90% do tempo em cem mil linhas, com *throughput* que varia por mais de uma ordem de grandeza entre SGBDs. Essa dispersão reflete o modelo de dados de cada banco e a forma de gravação idiomática correspondente — o MongoDB e o Redis, orientados a inserções em massa simples, lideram; o Cassandra e o Neo4j, que na configuração deste estudo aplicam respectivamente escrita replicada e `MERGE` com verificação de unicidade, ficam atrás. Trata-se de uma manifestação, na etapa de importação, das mesmas diferenças de paradigma que a literatura observa na etapa de consulta (MUNISWAMAIAH; AGERWALA; TAPPERT, 2023; LANDUYT et al., 2023).

Os dois modos de importação são equivalentes em custo (QP2), o que confirma, do ponto de vista de desempenho, a equivalência de resultados entre eles já assegurada por teste (seção 4.5.2). Quanto à escalabilidade (QP3), o tempo cresce de forma aproximadamente proporcional ao volume, governado pela fase de escrita, enquanto a memória segue o modelo de execução escolhido.

O maior ganho de engenharia está nos dois eixos de desempenho. A estratégia otimizada acelera a importação em cerca de $9\times$ (QP4), pela vetorização da conversão

de tipos e pela escrita em lote. O modo de fluxo reduz o pico de memória em $12{,}5\times$ em cem mil linhas (QP5), a um custo de tempo de apenas $+10\%$ nesse ponto — e o achatamento do pico com o volume (QP6) confirma que o fluxo importa com memória **constante** no tamanho do arquivo. É esse último resultado que sustenta a principal afirmação de projeto da ferramenta: a de importar arquivos maiores que a memória disponível, algo impossível no modo de materialização.

5.4.1 Ameaças à Validade

Os resultados devem ser lidos à luz de algumas limitações. Quanto à **validade de construção**, os tempos carregam incerteza da ordem de 20% (seção 5.1.2), de modo que diferenças menores que isso não foram interpretadas como significativas; o pico de memória, muito mais estável ($0{,}1\%$), não sofre dessa limitação. A separação em campanhas pareadas para tempo e memória evita a distorção que a instrumentação de memória introduziria nas fases, mas impede a leitura das duas métricas de uma mesma execução.

Quanto à **validade externa**, todos os experimentos correram em uma única máquina, sob Windows, com os cinco SGBDs em contêineres partilhando 8 GB de memória física. Esse ambiente não representa uma implantação de produção, com bancos dedicados e ajustados. O caso mais sensível é o do Cassandra, cujo *throughput* de escrita (912 linhas/s) reflete um *heap* de 1 GB em uma máquina virtual apertada — uma restrição de **ambiente**, não uma propriedade intrínseca do SGBD; um Cassandra provisionado adequadamente reduziria substancialmente a sua parcela na fase de escrita. Quanto à **validade interna**, o gerador determinístico e a limpeza dos bancos antes de cada repetição isolam o efeito medido, mas o volume avaliado (até cem mil linhas) é moderado, e não se explorou o efeito de caches quentes nem de execuções concorrentes. Aprofundar o caminho de escrita do Cassandra e estender o estudo a volumes maiores figuram entre as atividades futuras (Capítulo 6).

6 ATIVIDADES FUTURAS

Concluídas a implementação da ferramenta e a sua avaliação de desempenho, esta seção reúne as atividades que dão continuidade ao trabalho. Elas dividem-se entre a ampliação da forma de uso, o aprofundamento da avaliação e a extensão das funcionalidades de importação.

- **Interface gráfica *desktop*.** A ferramenta é hoje operada por uma interface de linha de comando (Seção 4.1). O próximo passo desejável — e viável ainda dentro deste ciclo de trabalho — é uma interface gráfica que monte o comando de importação de forma assistida e permita visualizar e editar os dois arquivos JSON de configuração, reduzindo a barreira de entrada para usuários menos familiarizados com a linha de comando.
- **Aprofundamento da avaliação de desempenho.** A avaliação do Capítulo 5 cobriu volumes de até cem mil linhas em uma única máquina, com os cinco SGBDs em contêineres partilhando os mesmos recursos. Três extensões se destacam: medir volumes maiores, na ordem de milhões de linhas; aprofundar o caminho de escrita do Cassandra — a fase dominante do custo — para além da concorrência já empregada pela estratégia otimizada (Seção 4.5.1); e repetir os experimentos em um ambiente com SGBDs dedicados e ajustados, isolando o desempenho da ferramenta das restrições do ambiente de contêineres.
- **Suporte a outros formatos de entrada.** Aceitar arquivos TSV e planilhas Excel, além de CSV, reaproveitando o mesmo mapeamento declarativo já existente.
- **Operadores de filtro adicionais e políticas de coerção configuráveis.** Ampliar o vocabulário de seleção de linhas e permitir que o usuário defina como valores ambíguos ou malformados são convertidos aos tipos nativos de cada SGBD.
- **Testes de integração em ambiente conteinerizado.** A suíte de testes atual é majoritariamente unitária; prevê-se automatizar testes de integração contra os cinco SGBDs em `docker compose`, dentro de um fluxo de integração contínua (CI).
- **Generalização do núcleo de importação.** Estender a arquitetura de portas e adaptadores (Seção 4.5.2) a novos destinos — sistemas de mensageria, *data lakes* e outros SGBDs —, reaproveitando o orquestrador agnóstico de destino sem duplicar a lógica de importação.

7 CONSIDERAÇÕES FINAIS

Este trabalho apresentou a *PolyglotImportCSV*, uma ferramenta de importação declarativa que materializa um mesmo conjunto de dados em formato CSV em cinco SGBDs de paradigmas distintos — PostgreSQL, MongoDB, Cassandra, Redis e Neo4j — a partir de dois arquivos de configuração validados estaticamente. Partindo da fundamentação sobre persistência poliglota e do estado da arte em importação e modelagem de dados heterogêneos, o trabalho evoluiu da proposta a uma implementação funcional, avaliada empiricamente.

As principais contribuições são: (i) um formato de configuração JSON, separado em mapeamento e conexão e validado por JSON Schema antes de qualquer escrita, com dois modos de entrada — um arquivo por entidade (multifonte) e um único CSV combinado; (ii) o mapeamento declarativo de colunas com conversão nativa aos tipos de cada SGBD; (iii) uma camada de observabilidade e um subsistema de métricas que instrumentam cada fase da importação; (iv) dois eixos de desempenho — a estratégia de escrita otimizada, com vetorização e gravação em lote, e o modelo de execução em fluxo, de memória limitada; e (v) uma avaliação empírica com dados reais que quantifica o custo de importar para os cinco paradigmas e o ganho de cada eixo.

A avaliação (Capítulo 5) confirmou os objetivos traçados. A estratégia otimizada acelerou a importação em cerca de nove vezes sobre a linha de base ingênua, e o modo de execução em fluxo reduziu o pico de memória em mais de doze vezes no maior volume medido, mantendo-o constante em relação ao tamanho do arquivo — o que habilita a importação de arquivos maiores que a memória disponível. A equivalência dos dados gravados entre os dois modos de importação e entre os dois modelos de execução, assegurada por testes automatizados, permitiu tratar a escolha entre eles como uma decisão de custo, não de correção.

As limitações reconhecidas dizem respeito sobretudo ao ambiente de avaliação: os experimentos correram em uma única máquina, com os cinco SGBDs em contêineres partilhando os mesmos recursos, o que penalizou em especial a escrita no Cassandra — uma restrição do ambiente, e não uma propriedade da ferramenta. Além disso, a suíte de testes permanece majoritariamente unitária, sem integração contínua contra os SGBDs conteinerizados.

Como trabalho futuro (Capítulo 6), destacam-se uma interface gráfica que torne a ferramenta acessível a usuários menos familiarizados com a linha de comando, o aprofundamento da avaliação em volumes maiores e em ambientes com SGBDs dedicados, e a generalização do núcleo de importação a novos destinos. Consolida-se,

assim, a *PolyglotImportCSV* como artefato de pesquisa e de código aberto alinhado à literatura sobre persistência poliglota e modelagem de dados heterogêneos.

REFERÊNCIAS

CANDEL, C. J. F.; RUIZ, D. S.; GARCíA-MOLINA, J. J. A unified metamodel for NoSQL and relational databases. *Information Systems*, v. 104, p. 101898, 2022. Citado na página 28.

CHILLóN, A. H. et al. A generic schema evolution approach for nosql and relational databases. *IEEE Transactions on Knowledge and Data Engineering*, v. 36, n. 7, p. 2774–2789, 2024. Citado na página 28.

CHILLóN, A. H. et al. Propagating schema changes to code: An approach based on a unified data model. In: *Proceedings of the Workshops of the EDBT/ICDT 2023 Joint Conference (CoMoNoS)*. [s.n.], 2023. (CEUR Workshop Proceedings, v. 3379). Disponível em: <https://ceur-ws.org/Vol-3379/CoMoNoS_2023_id251_Alberto_Hernandez_Chillon.pdf>. Citado 2 vezes nas páginas 13 e 28.

COOPER, B. F. et al. Benchmarking cloud serving systems with YCSB. In: *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*. [S.l.]: ACM, 2010. (SoCC '10), p. 143–154. Citado 2 vezes nas páginas 58 e 59.

DBCROSSBAR: copy tabular data between databases, CSV files, and cloud storage. 2024. <https://www.dbcrossbar.org>. Ferramenta de código aberto em Rust para migração e conversão de esquemas. Citado 2 vezes nas páginas 27 e 28.

DUGGAN, J. et al. The BigDAWG polystore system. *ACM SIGMOD Record*, v. 44, n. 2, p. 11–16, 2015. Citado na página 28.

FORD, N. *The Productive Programmer*. [S.l.]: O'Reilly Media, 2008. Citado na página 16.

FOWLER, M. *PolyglotPersistence*. 2011. Disponível em: <https://martinfowler.com/bliki/PolyglotPersistence.html>. Citado na página 16.

GLAKE, D. et al. *Towards Polyglot Data Stores: Overview and Open Research Questions*. 2022. Citado 3 vezes nas páginas 22, 28 e 29.

Hackolade. *Polyglot Data Modeling*. 2024. <https://hackolade.com/polyglot-data-modeling.html>. Modelagem conceitual com engenharia de esquema para múltiplos SGBDs. Citado na página 28.

HECHT, R.; JABLONSKI, S. Nosql evaluation: A use case oriented survey. In: *Proceedings of the 2011 International Conference on Cloud and Service Computing*. [S.l.: s.n.], 2011. p. 336–341. Citado na página 18.

KAUR, K.; SHARMA, S.; KAHLON, K. S. A middleware for polyglot persistence and data portability of big data PaaS cloud applications. *Computers, Materials & Continua*, v. 65, n. 2, p. 1625–1647, 2020. Citado na página 28.

KHINE, P. P.; WANG, Z. A review of polyglot persistence in the big data world. *Information*, v. 10, n. 4, p. 141, 2019. Citado na página 13.

KIEHN, F. et al. Polyglot data management: State of the art & open challenges. *Proceedings of the VLDB Endowment*, v. 15, n. 12, p. 3750–3753, 2022. Citado 4 vezes nas páginas 13, 22, 28 e 29.

LANDUYT, D. V. et al. A comparative performance evaluation of multi-model NoSQL databases and polyglot persistence. In: *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC)*. [S.l.]: ACM, 2023. (SAC '23), p. 286–293. Citado 2 vezes nas páginas 58 e 66.

MongoDB, Inc. *mongoimport — Database Tools*. 2026. <https://www.mongodb.com/docs/database-tools/mongoimport/>. Utilitário de importação de CSV/JSON/TSV para o MongoDB. Citado na página 26.

MUNISWAMAIAH, M.; AGERWALA, T.; TAPPERT, C. C. Comparison of SQL, NoSQL, and NewSQL database technologies. In: *2023 IEEE International Conference on Big Data (BigData)*. [S.l.]: IEEE, 2023. p. 6230–6232. Citado na página 66.

Neo4j, Inc. *Import — Operations Manual*. 2026. <https://neo4j.com/docs/operations-manual/current/tools/neo4j-admin/neo4j-admin-import/>. Importação em massa de CSV com `neo4j-admin database import`. Citado na página 27.

PEREIRA, F. d. O.; GUERRA, E.; ROSA, R. R. A framework model for supporting transparent polyglot persistence with a unified API and extensible for different database types. In: *Proceedings of the 27th International Conference on Enterprise Information Systems (ICEIS)*. [S.l.]: SciTePress, 2025. p. 109–120. Citado na página 28.

Redis Ltd. *Bulk Loading*. 2026. <https://redis.io/docs/latest/develop/clients/patterns/bulk-loading/>. Carga em massa via `redis-cli --pipe` usando o protocolo RESP. Citado na página 25.

ROY-HUBARA, N.; SHOVAL, P.; STURM, A. Selecting databases for polyglot persistence applications. *Data & Knowledge Engineering*, v. 137, p. 101950, 2022. Citado 2 vezes nas páginas 22 e 28.

SACHDEVA, S.; VASAVA, J. Plugging and playing with variety of data using multi-model database and polyglot persistence. In: *2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*. [S.l.]: IEEE, 2024. p. 1–8. Citado na página 28.

SADALAGE, P. J.; FOWLER, M. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. [S.l.]: Pearson Education, 2013. Citado 2 vezes nas páginas 13 e 22.

SILVA, H. A. B. d.; BORNIA, L. G.; MELLO, R. d. S. Um estudo sobre modelagem poliglota de dados. In: *Anais da Escola Regional de Banco de Dados (ERBD)*. [S.l.]: Sociedade Brasileira de Computação, 2024. p. 11–20. Citado 2 vezes nas páginas 13 e 28.

TAN, R. et al. Enabling query processing across heterogeneous data models: A survey. In: *2017 IEEE International Conference on Big Data (BigData)*. [S.l.: s.n.], 2017. p. 3211–3220. Citado 2 vezes nas páginas 22 e 28.

The Apache Software Foundation. *sstableloader*. 2026. <https://cassandra.apache.org/doc/latest/cassandra/managing/tools/sstable/sstableloader.html>. Ferramenta de carga em massa de SSTables no Apache Cassandra. Citado na página 26.

The PostgreSQL Global Development Group. *PostgreSQL Documentation: COPY*. 2026. <https://www.postgresql.org/docs/current/sql-copy.html>. Comando `COPY` para importação/exportação de dados, inclusive CSV. Citado na página 24.

YE, F. et al. A benchmark for performance evaluation of a multi-model database vs. polyglot persistence. *Journal of Database Management*, v. 34, n. 3, p. 1–20, 2023. Citado 3 vezes nas páginas 13, 16 e 58.

APÊNDICES

APÊNDICE A – JSON SCHEMAS DE CONFIGURAÇÃO NA FORMA SERIALIZADA

Este apêndice reproduz os dois documentos *JSON Schema* (rascunho 2020-12) tal como embutidos em `src/polyglotimportcsv/schemas/`. A estrutura de alto nível de ambos é descrita na seção 4.2; os fragmentos por SGBD são apresentados nas respectivas subseções do capítulo 4.

A.1 SCHEMA DE IMPORTAÇÃO (`IMPORT_CONFIG.SCHEMA.JSON`)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://polyglot-import-csv.local/schemas/import_config.schema.json",
  "title": "PolyglotImportCSV import configuration",
  "description": "Declarative mapping from named CSV sources to entities and
      relationships of one or more SGBDs. Connection settings live in the separate
      SGBD configuration.",
  "type": "object",
  "required": ["sources"],
  "properties": {
    "sources": {
      "type": "object",
      "minProperties": 1,
      "description": "Named data sources: a CSV path per entity, or a combined CSV
          whose column 0 holds each row's origin.",
      "additionalProperties": {
        "oneOf": [
          { "type": "string", "minLength": 1 },
          {
            "type": "object",
            "required": ["file", "origin_column"],
            "properties": {
              "file": { "type": "string", "minLength": 1 },
              "origin_column": { "const": true }
            },
            "additionalProperties": false
          }
        ]
      }
    },
    "postgres":  { "$ref": "#/$defs/postgresMapping" },
    "mongodb":   { "$ref": "#/$defs/mongoMapping" },
    "cassandra": { "$ref": "#/$defs/cassandraMapping" },
    "redis":     { "$ref": "#/$defs/redisMapping" },
    "neo4j":     { "$ref": "#/$defs/neo4jMapping" }
  },
  "additionalProperties": false,
  "$defs": {
    "columnSpec": {
      "type": "object",
      "description": "Leaf mapping from a CSV column to a destination field.",
```

```
    "properties": {
      "is_key": {
        "type": "boolean",
        "description": "Primary or merge key for the entity."
      },
      "csv_column": {
        "description": "CSV header name or 1-based data-column index when it
            differs from the JSON key.",
        "oneOf": [
          { "type": "string",  "minLength": 1 },
          { "type": "integer", "minimum": 1 }
        ]
      },
      "schema_column": {
        "type": "string",
        "minLength": 1,
        "description": "Destination field name when it differs from the JSON key."
      },
      "db_type": {
        "type": "string",
        "description": "SQL/CQL type for DDL generation (PostgreSQL, Cassandra)."
      }
    },
    "additionalProperties": false
  },
  "columnEntry": {
    "description": "Either a leaf columnSpec or a nested object (MongoDB
        subdocuments).",
    "oneOf": [
      { "$ref": "#/$defs/columnSpec" },
      {
        "type": "object",
        "minProperties": 1,
        "additionalProperties": { "$ref": "#/$defs/columnEntry" }
      }
    ]
  },
  "filter": {
    "type": "object",
    "description": "Row predicate applied to the full CSV before materialization
        .",
    "required": ["column", "operator"],
    "properties": {
      "column":        { "type": "string", "minLength": 1 },
      "operator": {
        "type": "string",
        "enum": ["==","!=",">","<",">=","<=","in","not_in","each"]
      },
      "value": {},
      "target_suffix": {
        "type": "string",
        "description": "With operator 'each', optional template; default is
            sanitized distinct value"
      }
    },
    "additionalProperties": false
```

```
    },
    "entity": {
      "type": "object",
      "description": "One logical target (table, collection, label, or Redis entity)
          .",
      "properties": {
        "columns": {
          "type": "object",
          "minProperties": 1,
          "additionalProperties": { "$ref": "#/$defs/columnEntry" }
        },
        "filters": {
          "type": "array",
          "items": { "$ref": "#/$defs/filter" }
        },
        "cassandra_partition": {
          "type": "array",
          "items": { "type": "string" },
          "description": "CSV/source column names forming the partition key (
              compound supported)"
        },
        "cassandra_cluster": {
          "type": "array",
          "items": { "type": "string" },
          "description": "Clustering columns (optional)"
        },
        "source": {
          "description": "Source name, or list of source names (union with _source
              per row). Omit to bind by the entity key name.",
          "oneOf": [
            { "type": "string", "minLength": 1 },
            { "type": "array", "minItems": 1, "items": { "type": "string", "
                minLength": 1 } }
          ]
        },
        "csv_columns": {
          "type": "array",
          "minItems": 1,
          "description": "Auto-map restriction: column names, 1-based indices, or
              ranges like \"1-5\". Only valid with auto-mapping.",
          "items": {
            "oneOf": [
              { "type": "string", "minLength": 1 },
              { "type": "integer", "minimum": 1 }
            ]
          }
        },
        "auto_map": {
          "type": "boolean",
          "description": "With 'columns' present, auto-map the whole source and let
              the manual entries override per column."
        }
      },
      "additionalProperties": false
    },
    "postgresMapping": {
```

```
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      },
      "relationships": {
        "type": "object",
        "additionalProperties": {
          "type": "object",
          "required": ["from","to","foreign_key"],
          "properties": {
            "from":           { "type": "string" },
            "to":             { "type": "string" },
            "foreign_key":    { "type": "string" },
            "references_key": { "type": "string" }
          },
          "additionalProperties": false
        }
      }
    },
    "additionalProperties": false
  },
  "mongoMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "cassandraMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
  },
  "redisMapping": {
    "type": "object",
    "required": ["entities"],
    "properties": {
      "entities": {
        "type": "object",
        "additionalProperties": { "$ref": "#/$defs/entity" }
      }
    },
    "additionalProperties": false
```

```
    },
    "neo4jMapping": {
      "type": "object",
      "required": ["entities"],
      "properties": {
        "entities": {
          "type": "object",
          "additionalProperties": { "$ref": "#/$defs/entity" }
        },
        "relationships": {
          "type": "object",
          "additionalProperties": {
            "type": "object",
            "required": ["from","to","type"],
            "properties": {
              "from": { "type": "string" },
              "to":   { "type": "string" },
              "type": { "type": "string" },
              "columns": {
                "type": "object",
                "additionalProperties": { "$ref": "#/$defs/columnSpec" }
              }
            },
            "additionalProperties": false
          }
        }
      },
      "additionalProperties": false
    }
  }
}
```

A.2 SCHEMA DE CONEXÃO (SGBD_CONFIG.SCHEMA.JSON)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://polyglot-import-csv.local/schemas/sgbd_config.schema.json",
  "title": "PolyglotImportCSV SGBD connection configuration",
  "description": "Connection settings for each SGBD. Lists which SGBDs are available
      ; the import configuration may only target SGBDs declared here.",
  "type": "object",
  "required": ["version"],
  "properties": {
    "version": {
      "type": "integer",
      "minimum": 1,
      "description": "Configuration format version."
    },
    "postgres":  { "$ref": "#/$defs/postgresConnection" },
    "mongodb":   { "$ref": "#/$defs/mongoConnection" },
    "cassandra": { "$ref": "#/$defs/cassandraConnection" },
    "redis":     { "$ref": "#/$defs/redisConnection" },
    "neo4j":     { "$ref": "#/$defs/neo4jConnection" }
  },
  "additionalProperties": false,
```

```
"$defs": {
  "postgresConnection": {
    "type": "object",
    "properties": {
      "connection": {
        "type": "object",
        "properties": {
          "host":     { "type": "string" },
          "port":     { "type": "integer" },
          "database": { "type": "string" },
          "user":     { "type": "string" },
          "password": { "type": "string" }
        },
        "additionalProperties": false
      },
      "schema": {
        "type": "string",
        "default": "public",
        "description": "Target schema for created tables."
      }
    },
    "additionalProperties": false
  },
  "mongoConnection": {
    "type": "object",
    "properties": {
      "connection": {
        "type": "object",
        "properties": {
          "uri":      { "type": "string" },
          "database": { "type": "string" }
        },
        "required": ["uri","database"],
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  },
  "cassandraConnection": {
    "type": "object",
    "properties": {
      "connection": {
        "type": "object",
        "properties": {
          "hosts":            { "type": "array", "items": { "type": "string" } },
          "port":             { "type": "integer" },
          "keyspace":         { "type": "string" },
          "protocol_version": { "type": "integer" },
          "request_timeout":  { "type": "number", "exclusiveMinimum": 0,
                                "description": "Client-side request timeout in
                                    seconds (default 30); the driver's 10s default
                                     is short for a bulk load." }
        },
        "required": ["hosts","keyspace"],
        "additionalProperties": false
      }
```

```
      },
      "additionalProperties": false
    },
    "redisConnection": {
      "type": "object",
      "properties": {
        "connection": {
          "type": "object",
          "properties": {
            "host":     { "type": "string" },
            "port":     { "type": "integer" },
            "db":       { "type": "integer" },
            "password": { "type": "string" }
          },
          "additionalProperties": false
        }
      },
      "additionalProperties": false
    },
    "neo4jConnection": {
      "type": "object",
      "properties": {
        "connection": {
          "type": "object",
          "properties": {
            "uri":      { "type": "string" },
            "user":     { "type": "string" },
            "password": { "type": "string" },
            "database": { "type": "string" }
          },
          "required": ["uri","user","password"],
          "additionalProperties": false
        }
      },
      "additionalProperties": false
    }
  }
}
```

APÊNDICE B – O ARQUIVO DE DADOS `ECOMMERCE_JOIN.CSV`

Este apêndice reproduz, em forma tabular, o arquivo de dados do cenário e-commerce, `data/ecommerce/ecommerce_join.csv`, utilizado nas seções 4.3 e 4.3.3. O arquivo contém 32 linhas de dados e 39 colunas, e resulta da **junção** (*join*) de todas as coleções de dados da fonte — usuários, endereços, produtos, categorias, estoque, pedidos, seleções de produto e carrinhos de compra — em uma única tabela larga. Por isso ele é **esparso**: cada linha representa um evento da loja virtual, identificado pela coluna `action` (`stock`, `purchase`, `select_product` ou `add_to_cart`), e preenche apenas as colunas pertinentes àquele tipo de evento — as demais permanecem vazias. A Tabela 16 resume quais grupos de colunas cada ação preenche. Este arquivo é a entrada do **modo combinado** de importação; no **modo multifonte**, os mesmos dados são fornecidos como quatro arquivos CSV separados, um por origem (`ecommerce_stock.csv`, `ecommerce_purchase.csv`, `ecommerce_select_product.csv` e `ecommerce_add_to_cart.csv`).

Tabela 16 – Grupos de colunas do `ecommerce_join.csv` preenchidos por ação.

Grupo	**Colunas**	`stock`	`purchase`	`select_product`	`add_to_cart`
Evento	`action`, `timestamp`	•	•	•	•
Usuário	`user_id`, `user_name`, `user_email`	•	•	•	•
Endereço do usuário	`street`, `neighborhood`, `state`, `country`, `zip_code`	•	•	—	—
Pedido	`order_number`, `order_date`, `order_status`	—	•	—	—
Contraparte da venda	`traded_with`, `trader_street`, `trader_neighborhood`, `trader_state`, `trader_country`, `trader_zip_code`	—	•	—	—
Avaliação e pagamento	`comment`, `rating`, `payment_method`, `quantity`	—	•	—	—
Produto	`product_id`, `product_name`, `product_variant`, `product_brand`, `product_description`, `product_image`	•	•	—	—
Categoria	`category_id`, `category_name`	•	•	—	—
Estoque	`quantity_available`, `price`, `last_restock_date`	•	•	—	—
Seleção de produto	`selected_product_id`, `suggested_product_count`	—	—	•	—
Carrinho	`shopping_cart_id`, `cart_product_id`, `cart_quantity`	—	—	—	•

Fonte: Elaborado pelo autor (2026).

Vários valores do arquivo seguem padrões regulares, o que permite apresentá-

los de forma resumida sem perda de informação. O produto `Product`N tem `product_id` N, pertence à categoria `Categoria`N (`category_id` N) e tem descrição e imagem padronizadas (`Description`M e `https://example.com/image`M`.jpg`). Cada usuário `user`N tem e-mail `user`N`@example.com`, rua `Street` N, `42` e um identificador alfanumérico (`user_id`) cujo início é dado na Tabela 17, junto com os demais dados de endereço. Nas tabelas deste apêndice, os usuários são referidos pelo nome (`user`N), tanto na coluna `user_id` quanto em `traded_with`.

Tabela 17 – Usuários do `ecommerce_join.csv` e seus dados de identificação e endereço.

Usuário	**`user_id` (início)**	**Bairro**	**Estado**	**País**	**CEP**
user1	`7fd7eee7-...`	ABC	CA	United States	12345
user2	`f246fb91-...`	DEF	SC	Brasil	84102149
user3	`09b3458a-...`	GHI	RS	Brasil	18057313
user4	`1fd3d09a-...`	JKL	TO	Brasil	19645442
user5	`0331463b-...`	MNO	RJ	Brasil	30097508
user6	`f4857da9-...`	PQR	SP	Brasil	92083256
user7	`a87df805-...`	STU	SP	Brasil	67065799
user8	`c11a1ef2-...`	VWX	SE	Brasil	18439703
user9	`d0416923-...`	YZ	AM	Brasil	18439666

Fonte: Elaborado pelo autor (2026).

Nas seções seguintes, o arquivo é apresentado em quatro blocos, um por ação. Cada tabela exibe as colunas com valores próprios daquele bloco; as colunas cujo valor é constante ou segue os padrões acima são descritas no texto que antecede cada tabela. Os valores e a ordem das linhas são exatamente os do arquivo original.

B.1 LINHAS COM `ACTION == STOCK`

As oito linhas de reposição de estoque preenchem os grupos de usuário, endereço, produto, categoria e estoque (21 das 39 colunas). Todas ocorrem em 02/11/2023 e, em cada uma, `last_restock_date` repete o instante do próprio evento (`timestamp`). A Tabela 18 apresenta os valores.

Tabela 18 – Linhas `action == stock` do `ecommerce_join.csv`.

Hora	Usuário	Produto	Variante	Marca	Categoria	Qtd.	Preço
03:30	user1	Product2	blue	Microsoft	Categoria2	2	50.5
06:45	user2	Product3	green	Google	Categoria3	1	52.07
09:15	user2	Product9	black	Pepsi	Categoria9	67	32.78
12:00	user8	Product8	white	Google	Categoria8	60	46.79
14:30	user9	Product4	yellow	Apple	Categoria4	82	55.94
17:15	user9	Product5	red	Samsung	Categoria5	57	34.21
20:00	user9	Product6	orange	Coca-Cola	Categoria6	64	51.98
22:30	user9	Product7	grey	Audi	Categoria7	1	100000

Fonte: Elaborado pelo autor (2026).

B.2 LINHAS COM `ACTION == PURCHASE`

As oito linhas de compra são as mais completas do arquivo (34 das 39 colunas): acrescentam aos grupos anteriores os dados de pedido, contraparte da venda, avaliação e pagamento. Em todas elas, `order_status` vale `Completed` e `quantity` vale 1; o registro do evento (`timestamp`) ocorre entre 15 e 19/11/2023, posterior à data do pedido (`order_date`), que inclui hora e fuso horário no arquivo original. O pagamento (`payment_method`) é `cash` nos pedidos `ORD1` e `ORD2` e `credit_card` nos demais. As colunas de produto, categoria e estoque trazem os dados cadastrais do produto negociado, tal como nas linhas de estoque (Tabela 18); o endereço do comprador e o do vendedor (colunas `trader_*`) são os da Tabela 17. A Tabela 19 apresenta os valores próprios de cada compra.

Tabela 19 – Linhas `action == purchase` do `ecommerce_join.csv`.

Pedido	Data	Comprador	Vendedor	Produto	Nota	Comentário
ORD1	03/11	user1	user2	Product2	2	Bad
ORD2	03/11	user2	user1	Product3	4	Bom
ORD3	05/11	user3	user9	Product4	3	Médio
ORD4	07/11	user4	user9	Product5	5	Ótimo produto
ORD5	09/11	user5	user9	Product6	1	Não gostei
ORD6	10/11	user6	user9	Product7	5	—
ORD7	11/11	user7	user8	Product8	1	Reprovado
ORD8	14/11	user8	user2	Product9	4	Bom, bonito e barato

Fonte: Elaborado pelo autor (2026).

B.3 LINHAS COM `ACTION == SELECT_PRODUCT`

As oito linhas de seleção de produto preenchem apenas a identificação do evento, o usuário e o grupo de seleção (7 das 39 colunas): o produto visualizado (`selected_product_id`) e a quantidade de sugestões exibidas (`suggested_product_count`), que vale 3 em todas as linhas. A Tabela 20 apresenta os valores.

Tabela 20 – Linhas `action == select_product` do `ecommerce_join.csv`.

Momento	Usuário	Produto selecionado	Sugestões
15/11 15:09	user1	11	3
16/11 03:11	user2	12	3
16/11 17:50	user3	22	3
17/11 06:05	user4	14	3
17/11 21:56	user5	14	3
18/11 10:09	user6	21	3
18/11 23:20	user7	33	3
19/11 15:18	user8	17	3

Fonte: Elaborado pelo autor (2026).

B.4 LINHAS COM `ACTION == ADD_TO_CART`

As oito linhas de adição ao carrinho preenchem a identificação do evento, o usuário e o grupo de carrinho (8 das 39 colunas). O identificador do carrinho (`shopping_cart_id`) segue o padrão `user:⟨user_id⟩:cart` — por exemplo, `user:7fd7eee7-. . . :cart` para `user1`. A Tabela 21 apresenta os demais valores.

Tabela 21 – Linhas `action == add_to_cart` do `ecommerce_join.csv`.

Momento	Usuário	Produto adicionado	Quantidade
15/11 15:17	user1	3	1
16/11 03:27	user2	2	1
16/11 18:08	user3	2	1
17/11 06:14	user4	2	1
17/11 22:11	user5	2	1
18/11 10:21	user6	2	1
18/11 23:34	user7	2	1
19/11 15:29	user8	2	1

Fonte: Elaborado pelo autor (2026).

APÊNDICE C – ARQUIVOS DE CONFIGURAÇÃO DO CENÁRIO E-COMMERCE

Este apêndice reproduz os arquivos de configuração do cenário de e-commerce utilizado no capítulo 4, tal como se encontram no diretório `data/ecommerce/` do repositório da ferramenta: o arquivo de importação no modo multifonte `import_config.json` (Listagem C.1), cujos fragmentos por SGBD ilustraram a seção 4.2; a sua variante para o modo combinado `import_config_combined.json` (Listagem C.2); e o arquivo de conexão `sgbd_config.json` (Listagem C.3). Junto com o arquivo de dados reproduzido no Apêndice B, eles constituem as entradas completas do exemplo, e neles podem ser conferidas as entidades, fontes e chaves que fundamentam as saídas materializadas da seção 4.3.2 e as contagens da Tabela 7.

C.1 ARQUIVO DE IMPORTAÇÃO (`IMPORT_CONFIG.JSON`)

```
{
  "sources": {
    "stock": "ecommerce_stock.csv",
    "purchase": "ecommerce_purchase.csv",
    "select_product": "ecommerce_select_product.csv",
    "add_to_cart": "ecommerce_add_to_cart.csv"
  },

  "postgres": {
    "entities": {
      "categories": {
        "source": "stock",
        "columns": {
          "category_id": { "is_key": true, "db_type": "BIGINT" },
          "category_name": { "db_type": "TEXT" }
        }
      },
      "products": {
        "source": "stock",
        "columns": {
          "product_id": { "is_key": true, "db_type": "BIGINT" },
          "product_name": { "db_type": "TEXT" },
          "product_variant": { "db_type": "TEXT" },
          "product_brand": { "db_type": "TEXT" },
          "category_id": { "db_type": "BIGINT" },
          "price": { "db_type": "NUMERIC(14,2)" }
        }
      },
      "inventory": {
        "source": "stock",
        "columns": {
          "product_id": { "is_key": true, "db_type": "BIGINT" },
          "quantity_available": { "db_type": "BIGINT" },
          "last_restock_date": { "db_type": "TIMESTAMPTZ" },
          "price": { "db_type": "NUMERIC(14,2)" }
```

```
      }
    },
    "orders": {
      "source": "purchase",
      "columns": {
        "order_number": { "is_key": true, "db_type": "TEXT" },
        "user_id": { "db_type": "TEXT" },
        "order_date": { "db_type": "TIMESTAMPTZ" },
        "order_status": { "db_type": "TEXT" },
        "payment_method": { "db_type": "TEXT" },
        "quantity": { "db_type": "BIGINT" },
        "product_id": { "db_type": "BIGINT" },
        "price": { "db_type": "NUMERIC(14,2)" },
        "comment": { "db_type": "TEXT" },
        "rating": { "db_type": "BIGINT" }
      }
    }
  },
  "relationships": {
    "product_category": {
      "from": "products",
      "to": "categories",
      "foreign_key": "category_id",
      "references_key": "category_id"
    },
    "order_product": {
      "from": "orders",
      "to": "products",
      "foreign_key": "product_id",
      "references_key": "product_id"
    }
  }
},

"mongodb": {
  "entities": {
    "product_catalog": {
      "source": "stock",
      "columns": {
        "product_id": {},
        "product_name": {},
        "product_variant": {},
        "product_brand": {},
        "product_description": {},
        "product_image": {},
        "price": {},
        "category": {
          "category_id": {},
          "category_name": {}
        },
        "stock": {
          "quantity_available": {},
          "last_restock_date": {}
        }
      }
    }
```

```
    }
  },

  "cassandra": {
    "entities": {
      "user_activity_log": {
        "source": ["stock", "purchase", "select_product", "add_to_cart"],
        "columns": {
          "user_id": {},
          "timestamp": { "schema_column": "event_time" },
          "_source": { "schema_column": "event_type" },
          "product_id": {},
          "order_number": {},
          "selected_product_id": {},
          "shopping_cart_id": {}
        },
        "cassandra_partition": ["user_id"],
        "cassandra_cluster": ["timestamp"]
      }
    }
  },

  "redis": {
    "entities": {
      "shopping_cart": {
        "source": "add_to_cart",
        "auto_map": true,
        "columns": {
          "shopping_cart_id": { "is_key": true }
        }
      },
      "user_session": {
        "source": "select_product",
        "columns": {
          "user_id": { "is_key": true },
          "user_name": {},
          "user_email": {},
          "timestamp": { "schema_column": "last_seen" }
        }
      }
    }
  },

  "neo4j": {
    "entities": {
      "User": {
        "source": "purchase",
        "columns": {
          "user_id": { "is_key": true },
          "user_name": {},
          "user_email": {}
        }
      },
      "Product": {
        "source": "stock",
        "columns": {
```

```
          "product_id": { "is_key": true },
          "product_name": {},
          "product_brand": {}
        }
      }
    },
    "relationships": {
      "PURCHASED": {
        "from": "User",
        "to": "Product",
        "type": "PURCHASED",
        "columns": {
          "order_number": { "is_key": true },
          "quantity": {},
          "price": {},
          "rating": {}
        }
      }
    }
  }
}
```

Listagem C.1 – Arquivo de importação completo do cenário e-commerce (`import_config.json`).

C.2 ARQUIVO DE IMPORTAÇÃO NO MODO COMBINADO (`IMPORT_CONFIG_COMBINED.JSON`)

No modo combinado, uma única fonte declara o arquivo largo `ecommerce_join.csv` com a marca `origin_column`, que instrui a ferramenta a tratar a primeira coluna de cada linha como a sua origem. O restante do arquivo — os mapeamentos de `postgres`, `mongodb`, `cassandra`, `redis` e `neo4j` — é **idêntico** ao do arquivo multifonte da Listagem C.1: as entidades continuam a referir-se às origens pelos mesmos nomes (`stock`, `purchase`, `select_product` e `add_to_cart`). A Listagem C.2 reproduz apenas o bloco `sources` que distingue os dois arquivos; as reticências indicam o trecho de mapeamento omitido por ser igual ao já apresentado.

```
{
  "sources": {
    "ecommerce": { "file": "ecommerce_join.csv", "origin_column": true }
  },

  "postgres":  { ... },
  "mongodb":   { ... },
  "cassandra": { ... },
  "redis":     { ... },
  "neo4j":     { ... }
}
```

Listagem C.2 – Bloco `sources` do arquivo de importação no modo combinado (`import_config_combined.json`).

C.3 ARQUIVO DE CONEXÃO (`SGBD_CONFIG.JSON`)

```
{
  "version": 1,

  "postgres": {
    "connection": {
      "host": "127.0.0.1",
      "port": 5432,
      "database": "ecommerce",
      "user": "postgres",
      "password": "postgres"
    },
    "schema": "public"
  },

  "mongodb": {
    "connection": {
      "uri": "mongodb://127.0.0.1:27017",
      "database": "ecommerce"
    }
  },

  "cassandra": {
    "connection": {
      "hosts": ["127.0.0.1"],
      "port": 9042,
      "keyspace": "ecommerce"
    }
  },

  "redis": {
    "connection": { "host": "127.0.0.1", "port": 6379, "db": 0 }
  },

  "neo4j": {
    "connection": {
      "uri": "bolt://127.0.0.1:7687",
      "user": "neo4j",
      "password": "password",
      "database": "neo4j"
    }
  }
}
```

Listagem C.3 – Arquivo de conexão completo do cenário e-commerce (`sgbd_config.json`).